BCSE498J- Project - II

TYPEMIND: FATIGUE MONITORING APPLICATION

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

22BCE3372 ADITYA AYAN SINGH

Under the Supervision of

Prof. Manjula R

Professor Grade 1

School of Computer Science and Engineering

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

April, 2026

DECLARATION

I hereby declare that the project entitled "TYPEMIND: FATIGUE MONITORING APPLICATION" submitted by me, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to VIT is a record of bonafide work carried out by me under the supervision of Prof. Manjula R, School of Computer Science and Engineering.

I further declare that the work reported in this project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore **ADITYA AYAN SINGH (22BCE3372)**

Date :

CERTIFICATE

This is to certify that the project entitled “TYPEMIND: FATIGUE MONITORING APPLICATION” submitted by ADITYA AYAN SINGH (22BCE3372), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering , is a record of bonafide work carried out by the student under my supervision during Winter Semester 2025-2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore **Signature of the Guide**

Date :

Internal Examiner **External Examiner**

Head of the Department

ACKNOWLEDGEMENT

I ADITYA AYAN SINGH (22BCE3372) am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering, for constant support and encouragement. The Dean's leadership and vision have greatly inspired me to strive for excellence. The dedication to academic excellence and innovation has been a constant source of motivation.

I express my profound appreciation to Dr. Boominathan P, the Head of the Department of Software Systems for insightful guidance and continuous support. The expertise and advice have been crucial in shaping the direction of my project. The commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. The constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project supervisor, Prof. Manjula R, School of Computer Science and Engineering, for dedicated mentorship and invaluable feedback. The patience, knowledge, and encouragement have been pivotal in the successful completion of this project. The willingness to share expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. This support has not only contributed to the success of this project but has also enriched my overall academic experience.

I extend my heartfelt thanks to all for the guidance and support received throughout this endeavor.

TABLE OF CONTENTS

EXECUTIVE SUMMARY

The widespread use of computers in academic, professional, and personal settings has increased concern about mental fatigue, as it can reduce attention, slow responses, and increase the likelihood of errors. Despite its importance, continuous fatigue monitoring remains difficult in practice because many existing approaches rely on intrusive tools such as physiological sensors or camera-based systems. These methods are often costly, uncomfortable, and unsuitable for everyday use. To overcome these limitations, this project introduces *TypeMind*, a non-intrusive system that detects mental fatigue through routine keyboard and mouse interactions.

The main objective of the project is to create a practical framework that can passively collect user interaction data, extract meaningful behavioural patterns, and classify the user as either relaxed or fatigued. The system captures keyboard and mouse events in real time and derives features such as key hold time, inter-key delay, typing speed, mouse movement speed, idle duration, and error-related behaviour. These indicators were chosen because they reflect changes in consistency, coordination, and interaction rhythm that commonly occur under fatigue.

To evaluate the system, behavioural data was collected under controlled relaxed and fatigued conditions and processed through a feature extraction pipeline. Several machine learning, deep learning, and ensemble models were then trained and compared using stratified cross-validation. Among the evaluated models, the Stacking Classifier achieved the strongest overall performance, showing that behavioural biometrics can effectively support fatigue recognition. The use of Variational Autoencoder-based augmentation further improved the robustness of the predictive model.

In addition to prediction, the project focuses on usability and interpretability. A Flask-based web interface was developed to support monitoring and visualization, while an LLM-based explanation module was included to provide user-friendly feedback and recommendations. Overall, TypeMind demonstrates a scalable, privacy-preserving, and cost-effective approach to real-time mental fatigue detection, with potential value for future human-centred systems that aim to improve wellbeing and productivity.

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

API	Application Programming Interface
AUC	Area Under the Curve
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DL	Deep Learning
DT	Decision Tree
ECG	Electrocardiography
EEG	Electroencephalography
EMG	Electromyography
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HCI	Human-Computer Interaction
HTML	HyperText Markup Language
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
KNN	K-Nearest Neighbors
LLM	Large Language Model
LR	Logistic Regression
LSTM	Long Short-Term Memory
ML	Machine Learning
RAM	Random Access Memory
RF	Random Forest
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
SVM	Support Vector Machine
VAE	Variational Autoencoder

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Due to technology, many people find themselves constantly interacting with computers. In fact, it is not unlikely to see individuals interacting with their computers in an office or even learning environment. Interacting with computers over extended periods of time results in cognitive overload or fatigue, whereby one's ability to focus, make decisions, retain information, and avoid mistakes is affected Grandjean (1979); Boksem et al. (2005).

However, there is a significant distinction between physical and mental fatigue, given that mental fatigue emanates from cognitive burden. Nevertheless, modern-day activities demand that individuals remain attentive, alert, and responsive to issues promptly. Therefore, these activities gradually exert pressure on an individual's cognitive ability, resulting in irritability, low productivity, inadequate decision-making, and eventually mental fatigue Lorist et al. (2005). The implications could be disastrous in sectors requiring precision, such as medicine, operational facilities, engineering management centers, and transport.

In cognitive science, research supports the detrimental effects of mental fatigue on executive functions, working memory, and sustained attention. As indicated by Boksem et al. Boksem et al. (2005), extended cognitive processing leads to a reduction in sustained attention and in the ability to detect errors. Event-related potential tests performed with the help of electroencephalography (EEG) demonstrate this finding. In another experiment carried out by Lorist et al. Lorist et al. (2005), mental fatigue was found to have an adverse impact on the control of cognitive processing, leading to reduced activity in the anterior cingulate cortex.

Hence, the problem of detecting fatigue in real time is significant and practical, and it comes under the domains of Human-Computer Interaction (HCI), cognitive science, and machine learning (Machine Learning (ML)). Moreover, the problem becomes more

challenging because fatigue is subjective in nature, considering the variations in people's experiences of fatigue with regard to factors such as workload, sleep, and related conditions.

1.1.1 Existing Approaches and Their Limitations

Technologies employed in fatigue detection systems in the past have mostly been based on physiological and observational technologies that have certain limitations regarding their application range. The current fatigue detection systems technology can be categorized into the following categories:

Physiological Monitoring Techniques consist of the usage of specialized sensors, which are able to detect the electric signals in the body. The usage of electroencephalography (Electroencephalography (EEG)) devices and sensors to detect brain wave changes due to the presence of fatigue has been proposed by Lal & Craig Lal and Craig (2003). On the other hand, the usage of electrocardiography (Electrocardiography (ECG)) sensors in monitoring the physiology due to the cognitive workload is in relation to heart rate and heart rhythms, while electromyography (Electromyography (EMG)) sensors are utilized to measure muscle fatigue. While such techniques are characterized by high accuracy, they are intrusive and very costly.

In contrast, camera methods employ cameras with specialized algorithms that process images for the purpose of identifying indicators of fatigue such as PERCLOS (prolonged eye closure), reduced blink rate, yawning, and head-nodding Ji et al. (2004); Zhang et al. (2017). Even though there is no form of body contact here, this system is a serious threat to an individual's privacy, owing to constant surveillance via videos. There are problems in terms of poor light adaptation, sensitivity to camera angles, and occlusions. Furthermore, its performance is poor when the user is wearing glasses or other face coverings.

Behavioral analysis is fairly a recent approach. The behavioral analysis approach involves analyzing natural behavior of the user based on timing of their keystrokes, mouse actions, and other behaviors to assess the state of mind of the user Vizer et al. (2009). Biometrics of behavior, which includes keystrokes and mouse activities, is an

interesting substitute for assessing the state of the user compared to physiological and vision biometrics. This is because the behavior is reflective of the mental state of the user.

Nevertheless, despite the improvements seen in all three systems, some problems still exist. Physiological systems are intrusive, burdensome, and costly; vision-based systems are privacy-intrusive and context-sensitive; and until now, all the behavioral systems studied have concentrated only on one form of behavior, whether it is through keyboards or mice, without taking advantage of the interaction between different forms of behaviors. Furthermore, existing systems provide little justification for their decisions on predictions.

1.1.2 Relevant Technologies and Concepts

The following are among the various components and notions that have been employed in building this system:

Keystroke dynamics is concerned with patterns such as holding time between keystrokes, the interval between keystrokes, keystroke speed, and keystroke errors that may be behavioral clues about the mental state of a user Monrose and Rubin (2000); Epp et al. (2011). According to studies, these keystroke patterns may alter due to cognitive load, fatigue, and stress Cao et al. (2020).

Features like mouse motion, velocity, acceleration, clicking action, and idling time are investigated under mouse dynamics to better assess the cognitive workload of a user Ahmed and Traore (2007); Sun et al. (2016). Reduced speed and consistency of mouse movement while performing complex cognitive tasks indicate an instance of mental fatigue.

The machine learning algorithms are used to extract the fatigue indicators from the behavioral biometric data through supervised algorithms such as Random Forest (Random Forest (RF)), Support Vector Machine (Support Vector Machine (SVM)), and Gradient Boosting techniques.

Deep learning enables learning of complicated patterns from behavioral data automatically. Some examples of deep learning approaches that can be particularly effective when relationships between variables are not easy to detect through feature engineering include long short-term memory (Long Short-Term Memory (LSTM)) Hochreiter and Schmidhuber (1997), convolutional neural network (Convolutional Neural Network (CNN)) LeCun et al. (1998), and transformer model Vaswani et al. (2017). On the other hand, generative models, like variational autoencoder (Variational Autoencoder (VAE)) Kingma and Welling (2014), prove to be particularly effective when there is not enough training data.

The use of large language models (Large Language Model (LLM)), like GPT Brown et al. (2020) and BERT Devlin et al. (2019), can enable an interpretation of model output in terms of natural language. In doing so, this provides an intuitive understanding of the underlying system.

1.1.3 Need for the Proposed System

As stated earlier, there is a need for a fatigue detection system that does not intrude, is affordable, and preserves privacy. A fatigue detection system that is prone to intrusion, difficult to use, expensive, and privacy-invading by using wearables or hardware is not feasible in practice. The solution to the above problem is fatigue detection software that uses standard input devices.

Additionally, notable research opportunities become limited due to the non-availability of sufficient datasets related to behavioral characteristics required for the identification of fatigue. In terms of fulfilling such requirements, the framework that is suggested for the project, named TypeMind, employs behavioral datasets generated by natural keyboard and mouse movements, thus eliminating the necessity of using specialized sensors, cameras, or self-reporting datasets. The TypeMind framework benefits from multimodal analysis of human behavior and uses different classifiers within ML and Deep Learning (DL) domains, as well as VAEs for augmenting data and LLMs for explaining decisions made by machines. TypeMind detects mental fatigue effectively while providing accurate and understandable results via a convenient and easy-to-use

web interface. The project not only fulfills the necessary societal requirements but also becomes an essential milestone for future studies in mental fatigue.

1.2 MOTIVATION

Such an initiative to develop an effective and workable method of detecting mental fatigue was inspired by some challenges that exist and indicate the importance of coming up with an efficient approach to detection of mental fatigue.

1.2.1 Problem Significance

It has been widely noted that mental fatigue is one of the key reasons behind making mistakes, low productivity, and safety concerns in many areas. Fatigued software engineers are more likely to make errors while writing code and introduce some flaws into it; fatigued doctors may find themselves unable to make correct diagnoses; fatigued drivers increase chances of getting involved in road accidents Lal and Craig (2003). The issue of a decrease in attentiveness and awareness is not only detrimental to an individual's condition, but also to the wellbeing and safety of other people in specific situations. For instance, the problem is especially acute in hospitals, engineering/operations control rooms, transport, and other places where any lack of concentration may cause serious harm.

1.2.2 Practical Motivation

Because of the presence of phenomena like telecommuting, online learning, and other computer-oriented behaviors, almost all individuals now spend a great deal of time in front of computers, trying to involve themselves in activities that require mental exertion, which lead to fatigue. There are many individuals who spend eight or more hours a day working on a computer and do not take breaks nor manage their fatigue.

A system capable of identifying the behavior patterns in a passive mode and alerting the user in case it finds any indication of fatigue will make an effective wellness device, which apart from instilling positive health behaviors in the workers, will ensure that no accident due to fatigue occurs without disrupting the workflow of the individual. The use of wearables and physical devices for monitoring purposes has been determined to

be intrusive, cumbersome, costly, and compromising on privacy. Therefore, a software solution without the need for wearables will be more practical than the conventional ones.

1.2.3 Academic Motivation

From the perspective of science, the research efforts made in the scope of this project represent yet another milestone in an actively evolving field in relation to the intersection of behavioral biometrics, ML, and cognitive state studies. In particular, the insufficient number of academic papers has been observed due to the lack of a sufficiently sized behavioral dataset to estimate fatigability. In turn, the absence of advancements in this sphere may impede further developments in other fields, including cognitive psychology, HCI, or ML. However, the current project relies on different kinds of input data, different machine learning algorithms, and advanced techniques of data augmentation and explainability of language models.

1.2.4 Technical Interest

This project entails numerous technical challenges that need to be overcome, including acquiring, analyzing, and processing high-frequency event data almost in real time, extracting features from these data, training different types of models, and integrating both generation and language models. Overcoming these challenges necessitates understanding of multiple fields, such as signal processing, ML, software engineering, and HCI.

1.2.5 Innovation

The TypeMind architecture makes advancements from the current solutions by considering both keystrokes and mouse activity for creating a comprehensive representation of user behavior rather than using only one type of behavioral data. Moreover, it integrates classical machine learning techniques, deep learning algorithms, and ensemble learning frameworks within a single architecture, allowing for better evaluation and tuning processes. In order to solve the challenge of the shortage of behavioral data, the system leverages VAE-based augmentation to generate extra data points for the training process. Another notable contribution of the solution is utilizing LLM-based expla-

nations to present prediction results in an easily interpretable manner, which increases user confidence in the proposed framework. Lastly, the framework provides a convenient web interface to observe fatigue metrics and alerts in real time.

1.3 SCOPE OF THE PROJECT

In terms of the scope of the project, this would include all the processes in the pipeline that are needed to create a system which can actually detect mental fatigue in a meaningful, useful and ethical way. In this part, we will be describing the scope limits of TypeMind, particularly in terms of functionality, nonfunctionality, technology, and assumptions.

1.3.1 Functional Scope

The software allows for real-time data collection in terms of collecting information regarding keyboard and mouse interactions of a user while using the computer. This is accomplished through Python scripts that capture key down events, key up events, clicks, cursor movements, and scrolling actions performed by the users. Such actions form the basis of behavior data collected for the purpose of the study.

The system further carries out feature extraction for the conversion of the raw logs into behavioral indicators. In this process, any noise and inconsistencies within the timestamp data are removed, while the interaction logs undergo normalization, statistical analysis, and segmentation. A total of 24 features are obtained from this process, including those related to keystroke dynamics, mouse dynamics, idle time, and errors like backspace or deletion.

Other important aspects of the system include the creation of models for detecting fatigue using machine learning. In this work, five models of traditional machine learning (ML), namely RF, SVM, K-Nearest Neighbors (K-Nearest Neighbors (KNN)), Decision Tree (Decision Tree (DT)), Logistic Regression (Logistic Regression (LR)), as well as three models of deep learning (DL), namely Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN), and Transformer are used. Two additional models are created based on Voting and Stacking classifiers.

There is also an option to include a data augmentation phase utilizing VAE. This phase helps to boost the volume of data used for training purposes by creating additional samples. To help the users interpret the results of predictions more easily, the LLM model is used to produce explanations in plain language. This means that apart from prediction results, users can receive personalized recommendations based on the detected state of fatigue.

Furthermore, the system also provides an easy-to-use graphical user interface (Graphical User Interface (GUI)) via the web to monitor the process in real-time. The fatigue metrics along with other output results can be viewed via this interface.

1.3.2 Non-Functional Scope

The system should react quickly to provide timely fatigue feedback. This feature is crucial for the system, as its usefulness directly relates to its ability to reflect the user's state. Another key aspect is scalability due to the modular nature of the system, which makes it easily extendable to accommodate additional users and functions in the future.

The importance of privacy, security, and usability cannot be overlooked in the system's design. The behavioral information should be ethically processed, stored in a local server, anonymized, and encrypted using the user's permission. The sensitive text entered by the user is not stored, thus minimizing the risk of any breach of privacy. Considering the frequent nature of the application, its online interface must be simple.

Furthermore, the system needs to be reliable and maintainable. It needs to continue functioning correctly even under changing circumstances, such as long periods of inactivity or rapid typing, without resulting in serious bugs or instability. Furthermore, a modular system will make maintenance easier in the future.

1.3.3 Technical Scope

TypeMind is primarily developed using Python 3.x, since it provides an excellent platform for data manipulation, machine learning, deep learning, and fast prototyping. In order to gather the behavioral data, TypeMind utilizes the `pynput` package to record

keyboard and mouse events. Post-gathering, the data is processed using Pandas and NumPy libraries to extract relevant behavioral information from user interactions.

For the model building process, the project employs Scikit-learn for performing machine learning tasks like classification Pedregosa et al. (2011), and PyTorch is used to build deep learning models like LSTM, CNN, and Transformers Paszke et al. (2019). In cases where data synthesis is required, the project makes use of a customized VAE. Also, an LLM is connected via an Application Programming Interface (API) for providing understandable explanations and recommendations. As far as the application component is concerned, the web-based application architecture is enabled by Flask Grinberg (2018), and data visualization is carried out using Plotly and Matplotlib.

1.3.4 Project Boundaries

The following are some of the practical assumptions under which the project is implemented.

First, there is an assumption that users will use both keyboard and mouse while collecting behavior patterns. There is an assumption that the users' fatigue labels are collected under circumstances where the state of the users can be easily identified. Moreover, the users will have consented to the recording of behavioral data. Finally, no sensitive text entered by users is collected.

Nonetheless, there are certain constraints associated with the proposed solution. Firstly, the solution only accommodates a single user, and secondly, fatigue is considered to be a two-state variable rather than a continuous one. The efficiency of the model relies on the quantity and quality of training data tailored to individual users. The training process for deep learning algorithms is computationally intensive. Furthermore, offline models cannot be implemented due to the reliance on LLMs.

1.3.5 Testing and Evaluation

Controlled experiments are performed all along the development to check the validity and reliability of the models, but how user acceptance and platform efficiency are affected is yet to be investigated.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

A literature review is an examination of previous research studies that can be used in a particular problem domain. Concerning the current project, the literature review will be conducted within the context of previous research on the problem of detecting mental fatigue, behavioral biometrics, mouse/keyboard typing dynamics, using machine learning techniques to classify cognitive states, and so forth. The objective of conducting the literature review will be to identify the approaches currently available and to highlight any potential gaps.

2.1.1 Mental Fatigue: Definition and Impact

The concept of mental fatigue is another subject area that received ample attention from researchers working in the areas of cognitive psychology and workplace safety. One such researcher was Grandjean Grandjean (1979), who presented one of the earliest comprehensive models of mental fatigue in an industrial setting. This model differentiated between mental and physical fatigue and highlighted the role of sustained cognitive activity in causing decreased productivity. The negative effects of mental fatigue on attention were further validated experimentally by Boksem et al. Boksem et al. (2005).

Lorist et al. took the discussion forward. Lorist et al. Lorist et al. (2005) examined brain functioning mechanisms in cognitive control while experiencing mental fatigue. From their findings, there is reduced activity in the anterior cingulate cortex, which is a brain region involved in detecting errors and conflicts, which shows that mental fatigue interferes with the brain's ability to control cognitive functions. Finally, Desmond and Hancock Desmond and Hancock (2009) classified fatigue into active and passive processes, highlighting that active fatigue can cause fatigue just as quickly as passive fatigue, which involves monitoring.

2.1.2 Physiological Approaches to Fatigue Detection

The physiologically-based approaches had been considered prevalent in the sphere of recognizing fatigue for many years already. Lal and Craig Lal and Craig (2003) provided an overview of several methods of psychophysiological detection of fatigue in drivers. Among the methods mentioned, EEG was indicated as the best indicator of fatigue due to its effectiveness of showing low activity of alpha and theta brain waves in association with impaired cognitive processes. Nonetheless, EEG had been associated with a lot of disadvantages including the issues of using electrodes and high cost of application.

In their paper, Ji et al. Ji et al. Ji et al. (2004) devised an actual real-time system of vigilance monitoring for drivers that uses eye tracking technology. In their research work, Ji et al. employed the use of several cameras and computer vision technologies to record instances where there were eye closures (PERCLOS), which are signs of fatigue in drivers. Despite their technology being very effective in laboratory settings, it also presented some privacy challenges and generalization problems.

However, the algorithm devised by Zhang et al. Zhang et al. (2017) to detect sleepiness through facial expression recognition was found to be very accurate; however, its efficacy became significantly limited if the algorithm worked in varying light conditions or if the users wore glasses and/or any face ornamentation.

2.1.3 Behavioral Biometrics for Cognitive State Analysis

The measurement and analysis of behavior for the purposes of identification, verification, or even mental states are known as behavioral biometrics. In contrast to physiological biometrics, which include fingerprint recognition and iris recognition, behavioral biometrics utilize behaviors that individuals learn or develop over time, including keystroke behavior, mouse behavior, gait, and voice characteristics.

Bailey et al. Bailey et al. Bailey et al. (2014) defines multi-modal behavioral biometrics as Very effective for user recognition and authentication, combining the three modalities of keystroke dynamics, mouse and stylometry. The experiment resulted that

combining the behavioral modes could reach higher accuracy of user Double modality enabling recognition: two modality to do the recognition, and provide some efficiency gain in recognition.

2.1.4 Keystroke Dynamics

Keystrok-ing has been widely studied in the area of behavioral biometrics, for authentication of a user and identification between individuals. In fact, Monrose and Rubin Monrose and Rubin (2000) were the first to use keystrokes dwell and flight time as a feature for biometric authentication. They demonstrated the typing styles are sufficiently individual to separate the one from another while type ing the same text.

In a study conducted by Banerjee and Woodard Banerjee and Woodard (2012) different keystroke dynamics based biometric authentication schemes were reviewed. The schemes from articles were grouped based on the feature type, classification technique, and keystroke evaluation scheme. Three main discriminatory factors for keystroke dynamics based user authentication were identified.

Alongside verification studies, keystroke dynamics has been studied as an alternative method of measurement of cognitive and affective states. Epp et al. Epp et al. (2011) demonstrated the ability to distinguish between emotions of confidence, hesitation, nervousness, and relaxation using keystroke. This set the foundation for the possibility of using keystroke dynamics as a measure of mental fatigue.

Vizer et al. Vizer et al. (2009) explored how keystrokes or language features could be leveraged to automatically detect stress. Their preliminaryanalysis demonstration found statisticallysignificantpredictive power, in the form of correlations, between speed, pause count, and number of typing errors with reported stress, again demonstrating the potential of these features to contribute to a prediction of mental fatigue. To further extend this line of inquiry, Cao et al. Cao et al. (2020) developed a system to predict cognitive fatigue from keystroat characteristics.

2.1.5 Mouse Dynamics

The idea of Mouse Dynamics analysis is to extract distinct behavioral features from mouse movements, clicks and pauses. As defined by Ahmed and Traore Ahmed and Traore (2007), mouse dynamics is a biometric modality that had a number of behavioral features consisting of mouse speed, mouse acceleration, mouse movement angle, length of the mouse click and direction of mouse movement. The system was able to provide accurate continuous authentication of users.

Pusara and Brodley Pusara and Brodley (2004) used mouse actions and found that speed, distance and curvatures are useful discriminat results showed that mouse actions can be used in a complementary way to keystroke actions, as they are different activities performed by the users.

[MINOR] Page Layout
Rule: Right margin overflow in BODY_TEXT.
Expected: <= 523.276pt
Detected: 550.71pt
Confidence: 77.7%
Reason: Single line overflow.

Sun et al. Sun et al. (2016) investigated the relationship between mouse behavior and mental work load and attention. They found that mice are more likely to slow down, having more erroneous mouse movement in accordance with higher mental work load. Mouse behavior can be used as a passive indicator of mental fatigue. We have good evidence to justify mouse dynamics as one of the input in TypeMind.

2.1.6 Machine Learning for Fatigue Detection

The mechanism of learning from behavioral patterns and inferring mental states is an application of the machine learning. There are several classification methods have been employed to identify the fatigue of drivers.

Random forest Breiman (2001) algorithm is one of the ensembles where several decision trees are built and whose output predictions are combined by the means of voting process. Random forest algorithm has found wide application within the field of behavioral biometrics because of its resistance against overfit-ting, ability to perform in high-dimensional feature space and feature selection itself. Support Vector Machine classifier designed by Cortes and Vapnik Cortes and Vapnik (1995) as an algorithm8.

That finds the best hyperplane to separate a feature space into two classes [34]. Thanks to these properties, SVM classifier has shown good results in classifying keystroke

dynamics.

Chen and Guestrin Chen and Guestrin (2016) created the XGBoost framework. This is a fast gradient boosting algorithm which has been shown to return state of the art results on many classification data sets. Its regularization capabilities and support for missing data make it especially useful for noisy behavioral data.

The key work in ensemble learning is done by Dietterich Dietterich (2000) who showed that an ensemble of diverse classifiers would necessarily perform better than any one of those classifiers alone. Zhou Zhou (2012) also provided the theoretical framework for understanding the ensemble approach, outlining the requirements for optimal performance of ensemble learning.

2.1.7 Deep Learning Architectures

Deep learning models represent a powerful framework for sequence modeling and autonomous feature discovery from sequential data. Hochreiter and Schmidhuber Hochreiter and Schmidhuber (1997) proposed a long-short term memory (LSTM) model to address the gradient consumption problem encountered in regular Recurrent Neural Network (RNN). LSTM model will be most suitable for analyzing sequential behavior data because of the need to persist across long-term time dependence between keystrokes and mouse clicks.

The CNN structure was recommended by Le Cn et al. LeCun et al. (1998) and CNN in this structure has good performance on feature extraction by convolving on local regions. Even though CNN was designed for image recognition, they have been used for time series classifications well by classifying biometric behavior data. CNN has the ability to learn features hierarchy without feature engineering.

The Transformer model was proposed by Vaswani et al. Vaswani et al. (2017). The key idea of the model is to predict the next element in a sequence based on the relationship of preceding elements, where the core of the model is self-attention mechanism. Perhaps it is because of this hat performance of Transformers has been quite successful

in many applications of sequence modeling, such as natural language processing and time series forecast-ing. With the aid of

With the help of self-attention mechanisms, the model is able to capture long-range relations in its input sequences, therefore it is desirable for behavior pattern recognition.

2.1.8 Generative Models and Data Augmentation

Kingma and Welling Kingma and Welling (2014) introduced the VAE, a gen-erative method that builds a probabilistic distribution of the latent space and produces new samples from the reconstructions of randomly chosen points. The use of VAE across several tasks as a datageneration method to augment the data, single or multi-target, especially when the number of training samples is limited or unbalanced, is documented in literature Doersch (2016). In the context of behavioral biometrics, VAEs enable the creation of synthetic behavioral traces.

Was extensive and comprehensive review of various approaches to data augmentation for deep learning. Shorten and Khoshgoftaar Shorten and Khoshgoftaar (2019) reflect on the importance of augmentation to learning performance and the reduction of overfitting problems. Motivated by this research, we consider applying VAE based augmentation to the problem of limited training data.

2.1.9 Large Language Models for Explainability

What is next big application of LLMs is developing human interpretable explanations. Brown et al. Brown et al. (2020) have demonstrated zero-shot and few-shot learning capabilities of gigantic language models, delivering reliable, coherent paragraphs on any topic without large amounts of domain-specific data. In 2019, the team behind BERT Devlin et al. (2019), a bidirectional transformer-based model, won the top spot in multiple NLP benchmarks trained on vast quantities of text.

In topic of fatigue detection system, LLM can be used to translate the predications' numerical form into texts if users want to understand the prediction more intuitively. In detail, predictions an Numerical values of feature are translated into texts.

2.2 RESEARCH GAP

Despite the amount of research that has already existed on the one hand, there are still some important gaps in the literature as described above. It is these gaps that so many of the purposes that TypeMind aims to target:

- No Non-Invasive Approaches: Most state-of-the-art fatigue detection system relies on sensors (mainly physiological sensors) or camera-based detection which are all invasive and/or expensive. Few researches explore the fully utilization of keyboard/mouse interaction behavior.
- Single-Modal Approaches: most of the works that examine behavioral bio-metrics for cognitive states analysis either look at a single modality such as keystrokes or mouse move-ments.
- limited model choices: all prior pieces have only selected a few models for fatigue classifier on behavioral features.
- Invalid Application of Data Augmentation: Generation of datasets of behavioral biometrics with the help of generative models such as VAE for fatigue detection has not found its application till now.
- Explainability Not Available: the methods in the already proposed systems output binarized result or score without providing any understandable (human friendly) explanation. The novelty of this fatigue detection method is the LLM based explainability.
- (2)A complete End-To-End Real-time System: Very few try have been made in the literature to build an integrated real-time system encapsulating the complete data flow from data acquisition to explanations and visualization.

2.3 OBJECTIVES

The goals of the TypeMind project are the following:

1. Design and implement a passive data collection mechanism that logs keyboard and mouse interactions in real-time while using a computer normally.

2. Develop a complete feature extraction pipeline that computes relevant behavioral features from raw interaction logs, such as typing behaviors, mouse trajectories, idle times, and error metrics.

3. Train various machine learning algorithms, both traditional (RF, SVM, KNN, DT, LR) and deep learning models (LSTM, CNN, Transformer), as well as ensemble models, for binary classification of the user's mental state (relaxed or fatigued).

4. Implement a VAE data augmentation pipeline that produces artificial behavioral data to increase data diversity and robustness.

5. Implement a LLM-powered module that provides natural language explanations and personalized advice regarding the fatigue prediction.

6. Design a web application for real-time monitoring and visualization of the fatigue detection process.

7. Evaluate the system's performance using typical classification metrics, such as accuracy, precision, recall, F1-score, and Receiver Operating Characteristic (ROC)-Area Under the Curve (AUC).

2.4 PROBLEM STATEMENT

This increase in duration and strain in human-computer interactions has led to an increase in the detection of mental fatigue in humans. Mental fatigue adversely affects a human‘s concentration, decision making and productivity. Its signs go unnoticed until it harms the user in some way. Existing mental fatigue detection systems are based on physiological sensors or camera-based systems, which are invasive, expensive, intrusive and cannot be used long-term.

A critical requirement will be to develop a system that will automatically detecting mental fatigue in real time based on the behavioral data (using keyboard and mouse activity data, without invading the users from the privacy point of view, and at no cost). The system should be capable of extracting the behavioral characteristics, training a classifier, producing the explanations of the system predictions, and demonstrating the results to users in a warm way.

2.5 PROJECT PLAN

The projects develops according to a clear set up plan, which has been broken down to 6 phases. With milestones and deliverables defined for each phases. A brief projectplan is found in the Table 2.1.

Table 2.1: Project Plan and Timeline

Phase	Activity	Duration	Deliverables
1	Literature Review and Requirements Analysis	Weeks 1–3	Survey report, requirements document
2	Data Acquisition Module Development	Weeks 4–6	Keyboard and mouse logger, raw data files
3	Feature Extraction and Preprocessing	Weeks 7–9	Feature extraction pipeline, processed dataset
4	Model Training and Evaluation	Weeks 10–14	Trained models, evaluation reports
5	System Integration and Web Interface	Weeks 15–17	Integrated system, web dashboard
6	Testing, Optimization, and Documentation	Weeks 18–20	Test reports, final documentation

Begin the literature study and the requirement analysis. This phase looks into existing study about mental fatigue detection, behavioral biometrics and ML modeling. It defines the requirement of the software and makes out the initial architecture design.

Data gathering and feature extraction aims at creating the necessary modules to acquire mouse and keyboard events from the `pynput` package. During this phase key down/up events, mouse position, mouse click/tap and scroll events are logged and, along with their timestamps, stored into well structured Comma-Separated Values (Comma-Separated Values (CSV)) files.

Model training and evaluation We train several ML & DL models over the preprocessed data-set. We tune the ML hyperparameters using cross-validation and measure per-formance measures like accuracy, precision, recall, F1 and ROC-AUC. This step also involves designing and evaluating the VAE data augmentation module.

System integration, web interface development, testing, optimization and documentation are another key set of activities focused on synthesis of all modules into a final solution. Flask framework was used to develop web application that interfaces with users, handles real-time data monitoring, visualization, and explanation via LLM. The final phase also encompasses unit, integration and acceptance testing, performance tuning, and final documentation.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

In this chapter, the technical requirements for the Type-Mind solution are presented. The technical requirements consists of hardware and software requirements, feasibility study, as well as module requirements in details. There are four types of requirements listed; hardware requirements, software requirements, functional requirements and non functional requirements.

3.1.1 Hardware Requirements

The hardware demands placed by the TypeMind system on the computer are not high, owing to its philosophy of running on regular computers without the necessity of any dedicated hardware. The hardware requirements are shown in Table 3.1.

Table 3.1: Hardware Requirements

Component	**Minimum**	**Recommended**
Processor	Intel Core i5 (8th Gen)	Intel Core i7 (12th Gen)
RAM	8 GB	16 GB
Storage	5 GB free space	20 GB SSD
GPU	Not required	NVIDIA GTX 1060 or higher
Input Devices	Physical keyboard and mouse	Physical keyboard and mouse
Display	1280 $\times$ 720 resolution	1920 $\times$ 1080 resolution
Network	Optional for LLM	Broadband

However, a separate Graphics Processing Unit (GPU) is needed to train deep neural networks (LSTM, CNN, Transformer), but not necessary for inference or the execution of classical ML models. Everything regarding inference can be done efficiently on CPU-based computers.

3.1.2 Software Requirements

Table 3.2 gives an overview of the software prerequisites needed for the development and execution of the TypeMind system.

Table 3.2: Software Requirements

Software Component	**Version / Details**
Operating System	Windows 10/11, Ubuntu 20.04+, or macOS 12+
Programming Language	Python 3.10 or higher
Data Collection	pynput 1.7.6
Data Processing	Pandas 2.0+, NumPy 1.24+
Machine Learning	Scikit-learn 1.2+
Deep Learning	PyTorch 2.0+
Web Framework	Flask 2.3+
Visualization	Matplotlib 3.7+, Plotly 5.14+
LLM Integration	OpenAI API (GPT-3.5/4)
Integrated Development Environment (IDE)	Visual Studio Code (recommended)
Version Control	Git 2.40+
Web Browser	Chrome, Firefox, or Edge (latest version)

3.1.3 Functional Requirements

Functional requirements are the exact features that the TypeMind system should deliver. Table 3.3 shows these requirements.

3.1.4 Non-Functional Requirements

The non-functional requirements states the qualities about the system that is built. This consist of:

- **Performance:** The prediction pipeline to perform the whole processing from feature extraction to inference in less than 0.5 seconds in classical ML and less than 1.0 second in deep learning cases.
- **Privacy:** Behavioral data will be stored on the user‘s local machine. No keystroke data will be collected (how users type each character) - only the timing and event meta-data will be reported.

Table 3.3: Functional Requirements

ID	Requirement	Description
FR-01	Keyboard Event Logging	Capture all key press and release events with timestamps
FR-02	Mouse Event Logging	Capture mouse movement, click, and scroll events with coordinates and timestamps
FR-03	Data Preprocessing	Clean raw logs by handling missing values, removing duplicates, and filtering noise
FR-04	Feature Extraction	Compute 24 behavioral features from preprocessed interaction data
FR-05	Model Training	Train classical ML, DL, and ensemble models on labeled feature data
FR-06	Real-Time Prediction	Generate fatigue predictions from live behavioral data with minimal latency
FR-07	VAE Augmentation	Generate synthetic behavioral samples using a trained VAE model
FR-08	LLM Explanation	Produce natural language explanations for each fatigue prediction
FR-09	Web Dashboard	Display real-time fatigue status, feature trends, and prediction history
FR-10	Model Persistence	Save and load trained models and scalers for reuse

- **Scalability:** The modularization should make possible to create new functionalities not affecting the existing implementation.
- **Data acquisition:** The data acquisition function should not crash or pause during the entire period of more than four hours.
- **Availability:** The web dashboard should be accessible through any browser and should have no additional installation apart from the Python application.
- **Ease of Maintainability:** The software should adhere to modularity principles with well-defined separation of data acquisition, pre-processing, feature extraction, modeling, inferences and visualization functionalities.

3.2 FEASIBILITY STUDY

Feasibility analysis assesses whether the system can be developed and utilized successfully using the resorces and environent available. Due to the nature of this project,

for the feasibility analysis, the three main areas to consider are technical, economical and social.

3.2.1 Technical Feasibility

The hardware and software components of the TypeMind system are practical to implement as they use well-established approaches and technologies including `pynput`, scikit-learn, PyTorch, and Flask, and have been proven to work for data collection, model development and web deployment respectively. It is computationally practical to run the system on an ordinary computer, and several prior physical keystroke dynamics, mouse dynamics and behavioral fatigue detection Monrose and Rubin (2000); Epp et al. (2011); Ahmed and Traore (2007); Sun et al. (2016); Vizer et al. (2009); Cao et al. (2020) studies provide research towards the TypeMind method.

3.2.2 Economical Feasibility

The project will be economically viable as it is using mostly open source software, therefore, there are no license fee implications and development costs are minimized. There is no dedicated hardware needed for regular operation, and the only recurring cost is for the LLM Application Programming Interface (API), this could be managed by limiting the usage or mitigated even further by utilizing local open source solutions.

3.2.3 Social Feasibility

The project is socially feasible as the nature of the project allows it to be used by every day users in a practical, and privacy orientated way. The system can be run as a very simple Python program and accessed via a standard web browser. The schedules discussed are also realistic, as the system can be built up in a moduleated fashion, testing each set of modules as they are built. This allows the overall project to be manageable.

3.3 SYSTEM SPECIFICATION

The following describes the system specification of the TypeMind framework system architecture. The hardware and software implementation should run on a regular PC/Laptop with a physical keyboard and mouse, enough processing power, enough

memory and local storage, and optional internet connection for on-demand LLM explanations. A dedicated GPU can be useful during the deep learning training phase but the normal inference phase as well as inference of traditional machine learning models (if used) should also work on a Central Processing Unit (CPU) based system without problems.

3.3.1 Hardware Specification

The TypeMind hardware spec was designed with feasibility and availability in mind; the system should run on most general purpose personal computers, with no requirement for specialized sensors or dedicated biometric hardware. It should primarily require: a CPU for continuous event capture and inference, generous Random Access Memory (RAM) for data processing and model running, space for logs, datasets, and learned models, input hardware standard of a physical keyboard and mouse. Support for display, and optionally network access, will help the user monitor the output, and have easy access to the LLM explanations.

3.3.2 Software Specification

The software specification diagram describes the entire flow of data for TypeMind, from data collection to presenting the results. In TypeMind, the keyboard and mouse activities are tracked and recorded through the use of `pynput`, where resulting event logs are then stored in a standardized CSV format. The data preprocessing phase utilizes Pandas and NumPy to take out any anomalies in the data, then split the continuous stream of data into time windows suitable for use as input to machine learning algorithms. The next step extracts 24 behavioral features from the information in each window to make inferences about fatigue. These features are used to train classical machine learning, deep learning, and ensemble prediction models, and in an autoencoder module to increase the dataset when new data is needed.

The software specification outlines the inference and graphical user interfaces. Real-time feature vectors are supplied to trained models to produce low delay fatigued or relaxed predictions. The user can configure which model to run through the system configuration file. The web applications runs under the Flask module and uses Hyper-

Text Markup Language (HTML), Cascading Style Sheets (CSS), JavaScript, and Plotly to generate graphical plots. These user applications display fatigue state, behavioral trends, prediction histories, and LLM explanations in a accessible web form. The 24 behavioral features input to the system are listed in Table 3.4.

Table 3.4: Behavioral Feature Set

No.	Feature Name	Category	Description
1	avg_key_hold_time	Keystroke	Mean duration between key press and release
2	std_key_hold_time	Keystroke	Standard deviation of key hold times
3	avg_inter_key_delay	Keystroke	Mean time between consecutive key presses
4	std_inter_key_delay	Keystroke	Standard deviation of inter-key delays
5	typing_speed	Keystroke	Keys pressed per minute
6	typing_burst_count	Keystroke	Number of rapid typing sequences
7	avg_burst_length	Keystroke	Mean number of keys per burst
8	pause_frequency	Keystroke	Number of pauses exceeding 2 seconds
9	avg_pause_duration	Keystroke	Mean duration of detected pauses
10	backspace_rate	Error	Proportion of backspace/delete presses
11	error_correction_rate	Error	Frequency of backspace-then-retype sequences
12	avg_mouse_speed	Mouse	Mean cursor speed in pixels per second
13	std_mouse_speed	Mouse	Standard deviation of cursor speed
14	max_mouse_speed	Mouse	Peak cursor speed recorded
15	avg_mouse_acceleration	Mouse	Mean rate of change of cursor speed
16	total_mouse_distance	Mouse	Cumulative cursor distance traveled
17	click_frequency	Mouse	Number of mouse clicks per minute
18	avg_click_duration	Mouse	Mean time between press and release of a click
19	scroll_count	Mouse	Total number of scroll events
20	scroll_speed	Mouse	Mean scroll magnitude per event
21	idle_duration_total	Idle	Cumulative idle time (no activity $>$3 seconds)
22	idle_count	Idle	Number of idle periods detected
23	avg_idle_duration	Idle	Mean duration of individual idle periods
24	active_ratio	Idle	Proportion of active time to total window duration

CHAPTER 4

SYSTEM DESIGN

4.1 CHAPTER OVERVIEW

This chapter presents the architecture of TypeMind. TypeMind is a non-intrusive system that is able to detect when a person is experiencing mental fatigue using keyboard and mouse input. The architecture of continuous event collection, feature extraction and prediction, explanation, and user interface into a complete and effective process will be described. The aim of this chapter is to present the reasons why certain architectural choices were made to allow accurate, reliable, and usable prediction of mental fatigue.

The general design architecture is very modular. Each process, especially the significant ones, is implemented by a dedicated module, which is helpful to control the dependence among modules and to improve the system. The modular design has a great importance in a research-oriented application, since the components tend to change more frequently.

4.2 OVERALL SYSTEM ARCHITECTURE

The architecture design of the proposed system is shown in Figure 4.1. The entire structure is designed as a layered one from recording interaction to creating interpretable fatigue information, allowing us to keep a clean data pipeline and test each layer separately.

The layer of lowest level is the logging layer. It log all of the keyboard and mouse activities on computer at ordinary working process. It send out the logged activities to preprocessing layer for removing noise or distortion and B-spline fitting. The feature extraction layer will take the behavioral features out from the interaction log and pass it to the downward layer for decision.

The Explanation and Interface modules translate the number out-put of classification into feedback up, that is above the prediction layer. It is not only classification, but full

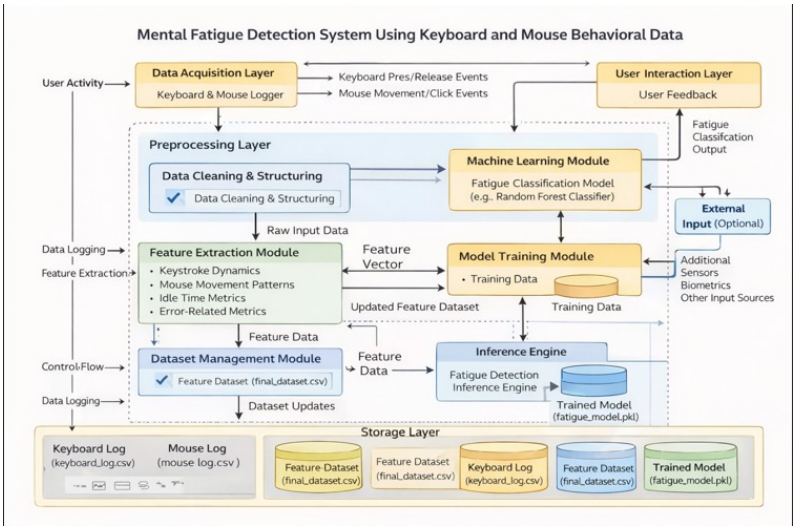

Figure 4.1: Overall system architecture of TypeMind

process of support to user, which means watching, analyzig and visualizing results.

4.3 DATA FLOW AND PROCESSING DESIGN

The flow of data through the system is depicted in the following figures, 4.2 & 4.3. Flow diagrams and flowcharts show the route that data takes from input to the prediction to output to the user. The figures show that the TypeMind system is run in stages, rather than in one large program.

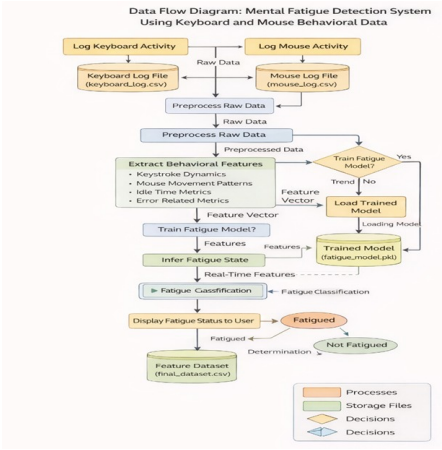

Figure 4.2: Data flow diagram of TypeMind

The first part in the sequence is normal working of the user with the keyboard-mouse set. The work is dumped into the logs with timestamp associated with every activity. In the preprocessing step, the duplicates are discarded, the incomplete pairs of events are

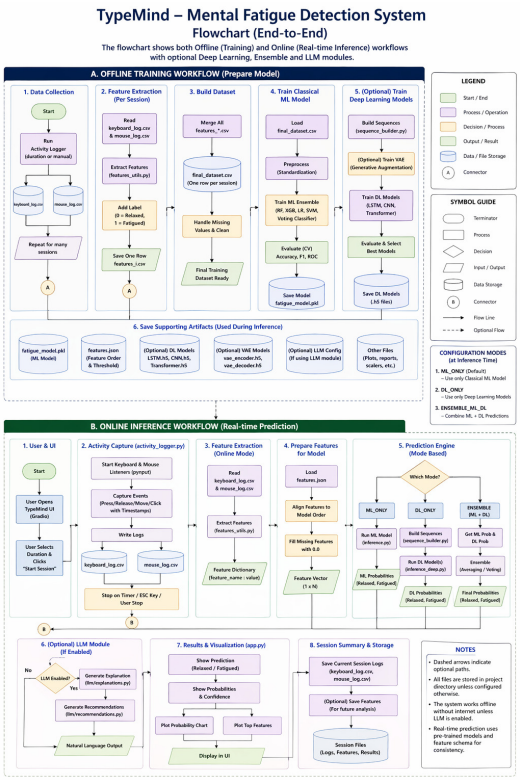

Figure 4.3: Operational flow chart of TypeMind

solved for and the sequence of events is categorized into several defined time windows (this is needed because fatigue behavior can be better viewed within a short duration).

During postprocessing, the feature extractor will measure some behavioral at-tributes, such as duration of key presses, interkey intervals, typing speed, mouse speed, idletime, as well as number of errors. They will be part of the vector at that time. Using this vector, the classifier will give its fatigue prediction which will be sent to the interface layer where it will be displayed to the user. If the state predic-tion is that the user is fatigued, the system will then generate an explanation of what should be done.

4.3.1 Core Functional Responsibilities

The logging process should run in the background continuously with minimal inter-ruption to the user. Its main objective is to monitor the user on the keyboard and mouse as effortlessly as possible by maintaining accurate time-stamp and structuring of the information obtained, it can be notdedown as the information collection process. The preprocessing process is then designed to prune unnessary information before analysis.

The feature extraction component acts as the bridge between the raw behaviors and the machine learning. To handle the large amount of individual measures of behavior, the feature extraction component combines measures of typing, pointing, idle, and correction behaviors into a larger set of behavioral data. This allows the model to be more robust, since fatigue may manifest differently in different individuals on different tasks. The state estimate is derived from the feature data by the prediction component, which becomes actionable through the explanation and dashboard components.

4.4 STATIC DESIGN OF SOFTWARE MODULES

In Fig. 4.4 is shown the class-level architecture of the proposed software. This structure of classes is selected for one reason, since the software will be developed according to the modular architecture. The aim of this class structure is to specify the function of each module of the software.

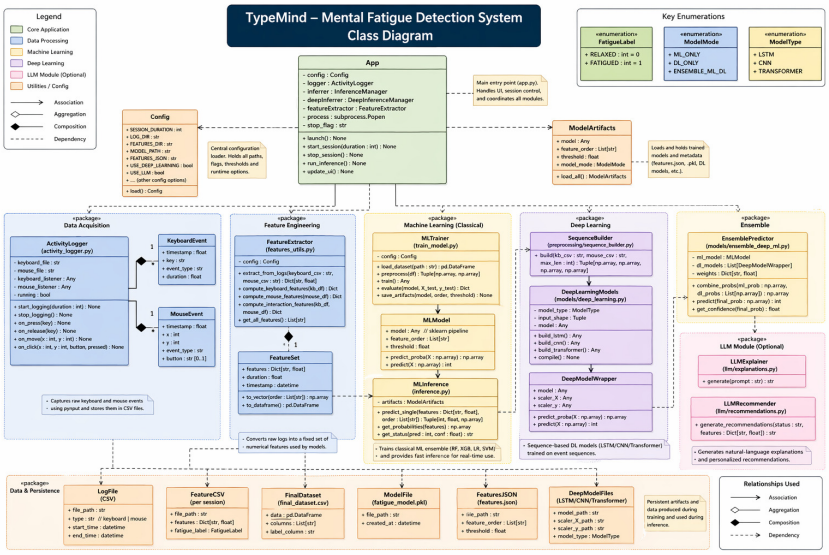

Figure 4.4: Class diagram of the major software modules

There are classes designed for separating the streams of the input streams from Keyboard and Mouse. Then, each stream can be logged in a separate stream to be processed parallelly. For preprocessing and feature extraction, data record is transfered to a suitable format for learning algorithms. The training component should be distinguished from the inference one for it moving on its own offline. Other components are used for creating explanation and web interface interaction.. The explanation component

converts prediction module output in user-friendly format, and the web app component control routing, display and user interactions.

4.5 DYNAMIC BEHAVIOUR OF THE SYSTEM

The application behavior during run time has been recorded in the form sequence diagram and State diagram, which can be viewed from Figure 4.5 and Figure 4.6. These two kinds of diagram demonstrate the modules within the application communicate and the state change of the application during a session.

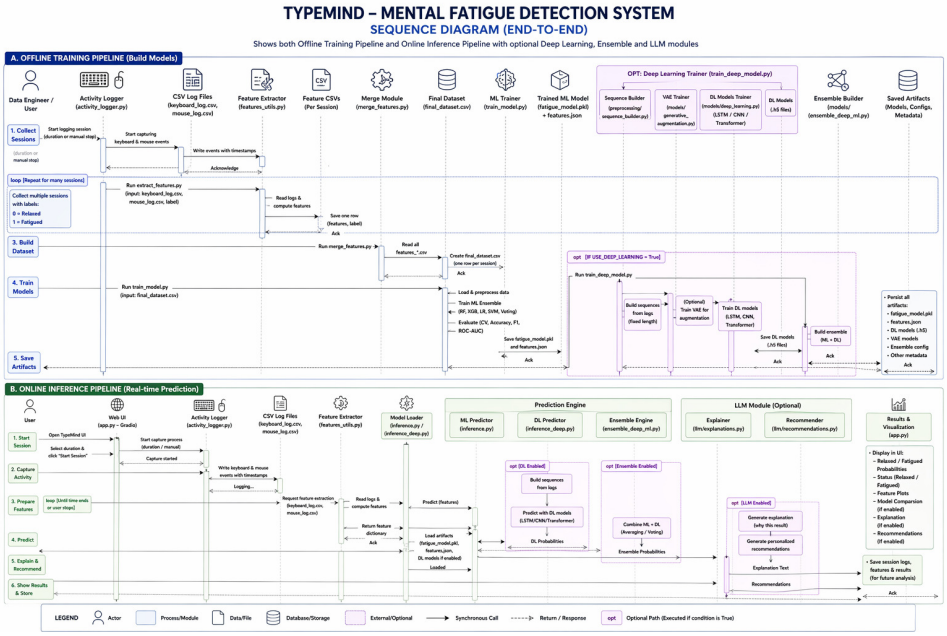

Figure 4.5: Sequence diagram of the runtime interaction flow

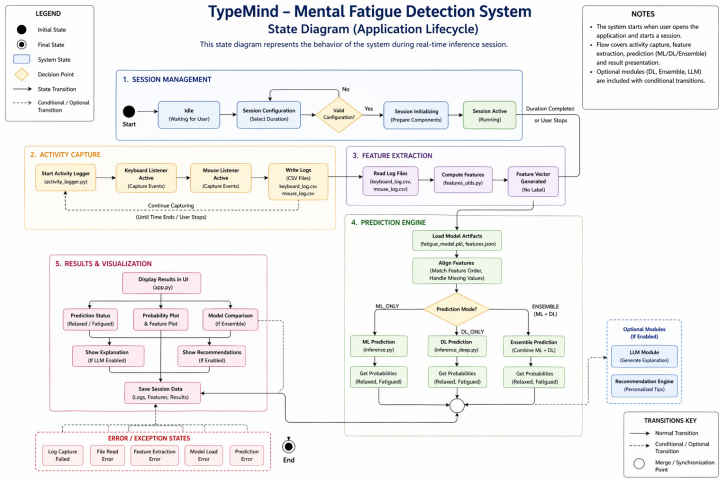

Figure 4.6: State diagram representing system behaviour during monitoring

During a normal session, the user is engaged in typical computer use while the logging components run as it were in the background, collecting data. Just before a period finishes, the new data is sifted and shaped to create a feature vector, which is then fed to the inference component for an up-to-date prediction and confidence score. Fatigue is also sensed by the user interface to activate the explanation component for output.

For the state machine, the process goes from the idle stage to the monitoring, preprocessing, the feature, the prediction and the reporting stage. When the report is created, the program predict again from the monitoring stage.

4.6 VISUALIZATION AND USER OUTPUT

System was built to display the predictions in a user friendly readable manner. The sample graph which would be used to plot the behavioral trend on every window is Figure 4.7. Using the trend plotting in the graph, user may get an idea of the reason a session would be given a fatigue label and also the researchers.

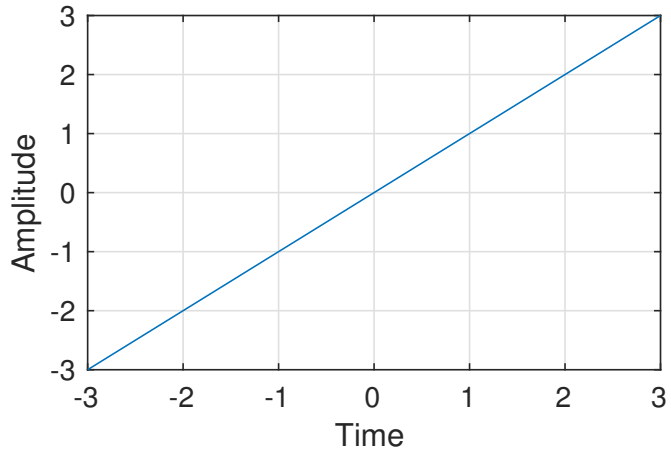

Figure 4.7: Sample visualization of behavioural trends over time

The dashboard is more than an indicator of results. It‘s a place where the forecasted state, also the index of certainty and behavioral measurements are combined in one entity. It allows having a more transparent system as it will be easier to believe the forecast once seeing its behavioral impact. All the records are stored to refine the model later.

Research too provides one more application of the visualization layer. It aids in comparing the session behavior over a span of time. Variation in total counts of idle time, error rate or typing style can be visually observed and patterns can be related back to the model.

4.7 DESIGN CONSIDERATIONS FOR PRACTICAL USE

Deployment concerns also influenced some of the design decisions for the system. Efficiency of background processes was one concern. Since the program will run while I working on the computer, it was desirable for the logging and predictions to be as light as possible. This influenced the decision to implement event-based logging and constant analysis windows.

Another problem was that of interpretability. It would be less useful if the user could not have any idea whether the system is on the right track to predict fatigue. This led to the requirement that the system would log the chosen behavior indicators involved in the result and display them alongside it.

The third concern involved whether the system would be expandable or grow-able. Again, because of the modular design, expansion is very simple due to the potential of more functionalities or classifiers being introduced into the system, or other commercial uses being discovered. Since the major components of the system were specified separately, any modifications could be carried out without affecting the entire code.

4.8 DESIGN RATIONALE

Several key decisions account for the design of TypeMind. First of all, the approach was developed following usual human interactions and not focused on sensors that could easily draw user's attention. Secondly, the fixed time sampling approach was used because the process of changes related to fatigue adapts to the slow changes and can be detected within short-term interactions. Thirdly, TypeMind was built in several ease modules so that later the application could be refined by improving specific processes involved in its development, e.g. Data preprocessing, feature engineering, model development, UI development.

Collectively, all the designs presented in this chapter constitute a comprehensive picture of the system. Architecture design reveals the layers of the system, workflow design shows the flow of information, class diagram details the internal structure of the system and behavioral diagrams demonstrate how the system operates during the run time. Together with the visualization design, the entire picture illustrates that the development of the TypeMind application was not confined to prediction engine design.

In conclusion, the design described this chapter has proved that accommodating the system with the machines ability of being simple and easy to use is achievable. It has created the possibility for a system to be accessed in real time and the consideration of the future features.

One good positive aspect of this design is its flexibility which allows its use for research and use. That means the same design can be utilized for feature exploration, comparison of different ML models, or for building a different explanation, with an insignificant modification to the design. That is why this chapter presents such a flexible design.

CHAPTER 5

METHODOLOGY AND TESTING

5.1 RESEARCH METHODOLOGY

This chapter describes the methodology by which the TypeMind solution was developed, designed and evaluated. This methodology comprises elements such as data collection, feature engineering strategies, modeling strategy as well as the testing methodology by which the solution was tested.

This experiment takes an experimental method where data relevant to behavior is collected in an environment designed for the existance of two level one mental states: relaxed and exhausted. The data then needs to beAnalyzed to find meaningful features of behavior. Several modeling techniques are then used to learn a classier for using the data to discriminate between the two states. These 4 steps are iterated in this process:

5.2 DATA COLLECTION PROTOCOL

Put differently, data of behavior observed in a controlled environment is taken by the experiment to simulate both a tired and a tired state. Asssumed used by participants standard PC or Laptop with the keyboard and mouse, with the TypeMind data gathering module operating in the background and recording everything mouse and keyboard does during the experiment session.

Each collection session is designed in a similar way. In the relaxed or baseline session, the subject is asked to do normal computer work such as pen and paper surveys, typing, browsing information etc. in the first work shift of the day at a time when the subject is well rested and alert. This will take between 30 minutes and 45 minutes and is referred to as class 0 representing the relaxed state. For the fatigued session the same or a similar task will be performed after a long period of full cognitive load (normally 3-4 hours) which causes the subject to feel mentally tired. This session also takes between 30 and 45 minutes and is referred to as class 1 representing the fatigued state. Following each session, the subject is asked to fill out a short self-report to verify the

way the subject perceives his state of mind and to compare this to the label selected using a common fatigue scale.

The data labels are determined based on the relaxed or fatigued condition of the session. The continuous interactions are followed by dividing them into equal segments of 5 minutes in duration and use the same label as the entire session. The summary of data collected is represented in Table 5.1.

Table 5.1: Data Collection Summary

Parameter	**Value**
Number of sessions (relaxed)	15
Number of sessions (fatigued)	15
Average session duration	35 minutes
Window size for feature extraction	5 minutes
Total feature samples (relaxed)	105
Total feature samples (fatigued)	105
Total dataset size	210 samples
Number of features per sample	24

5.3 FEATURE ENGINEERING

Feature engineering refers to the practice of deriving effective features, from interaction event logs, into a feature matrix for training machine learning models (other than the play-by-play features discussed in the previous section). In this work, the term covers raw features computation, feature transformation, and feature selection. For every time window, in each game, the 24 behavioral features introduced in Section 3 (Table 3.4) are computed using the techniques described in Section 4, and the raw output values are stored in a Pandas DataFramewhose every row corresponds to a time window and every column to a feature.

Prior to training, features are normalized to ensure a high quality data set and increased stability in training. Missing features are imputed (e.g., when there is no mouse movement on the screen) by replacing the values of the feature with the median for that feature and values beyond 3 standard deviations are Winsorized not to bias model

training. All features are normalized.

$$Xnormal = x \tag{5.1}$$

To measure the viscosity of the sample using the drop of sample which is the same size as the capillary tube drop and comparing the time to drop the same volume of sample in the glass tube to that of the water.

Where is the mean and the standard deviation on the training set. Also, the Pearson correlation coefficient is used to screen for pairs of highly correlated features (correlation 0.95) that may be eliminated for reducing multicollinearity.

Feature selection is done by averaging variable importance, based on random forest (RF) Breiman (2001), where a features importance is measured by its contribution to average impurity decrease in the nodes of the RF. Features such as the phone accessing due to either very low importance (¡1% of the maximum importance) or high importance (¿0.4), are considered feature selection. But in our experiments, all these potential features are selected. In order to give a less biased perspective, we also compute Gini Importance and Permutation Importance Chandrashekar and Sahin (2014). The most consistently important features are: average key hold time, typing speed, average inter-key delay, average mouse speed, backspace rate, idle duration total, pause frequency, and error correction rate.

5.4 MODEL DEVELOPMENT

Model development encompasses the training, implementation, and hyperparameter tuning of all classification models available in the TypeMind system.

5.4.1 Classical Machine Learning Models

Using a standard Python library 5 traditional ML models have been implemented Pedregosa et al. (2011). Also the hyperparameters of all models were tuned by using 5 folds of stratified cross validation on the training data set. The hyperparameter configuration of each model are shown in Table 5.2.

Table 5.2: Hyperparameter Configurations for Classical ML Models

Model	**Hyperparameters**
RF	n_estimators = 100, max_depth = 10, min_samples_split = 5, min_samples_leaf = 2, criterion = gini
SVM	kernel = rbf, C = 1.0, gamma = scale, class_weight = balanced
KNN	n_neighbors = 5, weights = distance, metric = minkowski, p = 2
DT	max_depth = 8, min_samples_split = 5, min_samples_leaf = 3, criterion = gini
LR	C = 1.0, solver = lbfgs, max_iter = 1000, class_weight = balanced

5.4.2 Deep Learning Models

Through PyTorch were the implementation of the three deep neural network model structures implemented. The three models take as input and 24 dimensional feature.

LSTM Architecture: The architecture of the LSTM is: two LSTM layers are stacked (with 64 neurons each), one dropout layer ($p = 0.3$), two fully-connected layers. The first one has 64 neurons and the second one has 2 neurons. The inputs are reshaped in a sequence form where each one is considered a time step and send to the LSTM network. Adam optimizer Kingma and Ba (2015) with $\eta = 0.001$ is used to train the LSTM network.

CNN Architecture: the convolution on the feature vector is done using the 1D CNN assuming it is a one-channel signal. In the architecture, the CNN used 2 convolutionnal layer with filter 32 and 64, kernel 3 with ReLU activation function followed by max pooling layer, then flatten layer, dropout layer with $p = 0.3$, and two fully connected layer (128>2). Batch normalization Ioffe and Szegedy (2015) is added after each convolutionnal layer.

Transformer Architecture: Several options are tested using a Multi-Head Self-Attention (MHA) method Vaswani et al. (2017) to model feature interactions. The

approach is to combine the attention mechanism with a feedforward structure. The architecture contains one embedding layer (for positional information), 2 Transformer encoder layers that use 4 attention heads with 64 dimensional embedding vectors, 2 Dense layers (64-2). Dropout rate of 0.2 Srivastava et al. (2014) in the architecture.

Table 5.3 summarizes the deep learning model configurations and training hyperparameters.

Table 5.3: Deep Learning Model Configurations

Model	**Hidden Units**	**Layers**	**Dropout**	**Learning Rate**	**Epochs**
LSTM	64	2 LSTM + 2 FC	0.3	0.001	100
CNN	32, 64 filters	2 Conv + 2 FC	0.3	0.001	100
Transformer	64 embedding	2 Encoder	0.2	0.0005	150

5.4.3 Ensemble Methods

There are two approaches used for ensembling the results from different base classifiers:

Voting Classifier: The best performing classical machine learning algorithms (RF, SVM, LR) build a soft voting ensemble. Since all classifiers predict class probabilities, the class label is chosen according to the weighed average of the input class probabilities Dietterich (2000).

Stacking Classifier: using RF, SVM, KNN and DT as base learners and LR as meta learner to build a two-level stacking ensemble. The out-of-fold predictions for the base learners, which are generated through 5-fold cross validation, will be input into the meta-learner Zhou (2012).

5.4.4 VAE Data Augmentation

Training of the VAE augmentation model is carried out on the already available dataset by learning the distribution of the data. The VAE is trained for 200 epochs using the ADAM optimization technique to minimize the ELBO loss function at a learning rate of 0.001. The VAE then produces synthetic samples using the following process:

1. Sampling latent vectors from the prior distribution: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

2. Passing the latent vectors through the decoder to generate synthetic feature vectors: $\hat{\mathbf{x}} = \text{Decoder}(\mathbf{z})$

3. Assigning labels to synthetic samples based on their proximity to real samples in the feature space using the nearest-neighbor approach.

The augmented data set is produced by concatenating the original data set and the newly synthesized instances. The effectiveness of the data augmentation process is measured by testing machine learning models using both the original and augmented data sets.

5.5 TESTING FRAMEWORK

The testing methodology is methodical, covering unit testing, integrative testing, system testing, performance testing and cross-validation. At the unit level, the target is to test whether the main sub-functions at each modules perform correctly, particularly to test if the keyboard and mouse events are correctly captured in the specified CSV format, whether the 24 features are correctly computed, if the preprocessing module preprocess correctly for the missing values, outliers and normalization, whether models can be successfully created and trained with sample data, and if the inference engine loads trained models correctly to make accurate predictions.

Then, integration testing verify that the different modules interact correctly. From the data pipeline (from event log to preprocessing to dataset generation) to the training pipeline (from the dataset loading, to feature scaling, to network training to validation to model persistence), the inference pipeline (from real-time data to model loading to predictions generation to LLM explanations to visual interface rendering), and the VAE pipeline (from the dataset loading, to network training, to synthetic samples generation, to data augmentation), the system is tested at the system level (all module are tested together as a whole using full data collection to predictions to visualization sessions). Further, the whole system is tested under boundary conditions (data collection sessions with only mouse data, only keyboard data, very short sessions, very fast input, and

extended idle periods. Furthermore, browser compatibility of the problem has been tested on Chrome Firefox and Edge.

Standard measures of classification performance Powers (2011); Sokolova and Lapalme (2009) are used to assess the effectiveness of the supervised learning. the accuracy roughly estimate the percentage of correctly classified samples:

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \tag{5.2}$$

such that I. When a, b, c,.. are symbols, T (x) is a total function binding on symbols a, b, c,. Like this, the symbol T (x) is a binding symbol. And the formula has an entry to a hiding operator named T (xx.).

Precision measures the proportion of predicted positives that are truly positive:

$$\text{Precision} = \frac{TP}{TP+FP} \tag{5.3}$$

Recall, or sensitivity, measures the proportion of actual positives that are correctly predicted:

$$\text{Recall} = \frac{TP}{TP+FN} \tag{5.4}$$

The F1-score is the harmonic mean of precision and recall:

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5.5}$$

Apart from the exceptions we have noted on the use of (5.4), the latter rule takes precedence over the others. For example, we have to consider the following:

The ROC-AUC value measures the discriminative capability of the classifier between the two classes. at each threshold. Where CR represents the true positive, CR the true negative. CR is the false positive, and CR the false negative. In our experiment, class 1 is the fatigued state (positive), class 0 is the relaxed state (negative).

In order to get a stable estimate of the models performance, all models are tested with a stratified k-fold which has $k = 5$, Kohavi (1995). This of course controls every fold

to have the same class equality as the full data set, which can be very important when a stable and equitable judgment is needed.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 DEVELOPMENT ENVIRONMENT

The TypeMind system has been developed and implemented in Python programming language version 3.10. TypeMind system has been developed using Visual Studio Code (IDE) and Git. Different modules of the TypeMind system have been implemented, which are discussed in the following sections.

6.2 DATA ACQUISITION MODULE IMPLEMENTATION

The data acquisition module involves using two logger threads working simultaneously for the keyboard and mouse inputs using the `pynput` library. The software architecture is configured in a way such that it runs in the background as a service, allowing data acquisition from the user's actions constantly.

The keyboard logger has two event listeners for the event of key presses and another for key releases. For every logged event, information on the key pressed, type of the event, and timestamp are collected. Both regular keys and special keys are covered. In order to ensure reliability of the logger, the event logging procedure is made thread-safe, thus avoiding corruption of the data in the case of concurrent events. Special keys also have their representations normalized to strings like `Key.shift` regardless of whether it was `Key.shift_l` or `Key.shift_r`. Lastly, events are accumulated in memory and written to the file either per 100th event or every five seconds, whichever comes first.

The mouse logger captures events related to movement, clicks, and scrolling. Mouse movement may happen with extremely high speed, therefore, rate limiting of the events is necessary in order to log mouse movement only after reaching a certain distance, which by default is set to be five pixels. Coordinate logging occurs in terms of absolute pixel values and in cases where several screens are used, the coordinates are normalized relative to the primary screen. In addition, mouse scrolling magnitudes in both dimensions are captured by the logger, as well as both directions of scrolling.

6.3 DATA PREPROCESSING IMPLEMENTATION

A preprocessing pipeline is utilized to convert the raw log files into data that are ready for analysis. While at the cleaning phase, common issues encountered in interaction logs include duplicate entries with identical keys, types of events, and timestamps due to repeated events and logging bugs. Missing complementary events, such as a key press without an accompanying key release event or vice versa, are handled by either predicting the missing event based on the average hold time of the key or discarding the entry. Timestamps are validated to ensure consistency, particularly being within the session limits and in the correct sequence, while timing heuristics are employed to filter system events, like auto-repeats of a key press.

With the data cleaning complete, the logs are split into equally-sized segments depending on the length of the session. Segmentation can be carried out relative to the beginning of the session and, if needed, allow overlapping of adjacent segments. In this project, Pandas' `pd.cut` function is used to sort the events into their respective bins based on their timestamps.

6.4 FEATURE EXTRACTION IMPLEMENTATION

The feature extractor module generates all the 24 behavioral features for each window based on efficient vectorization of Pandas and NumPy operations. For keystroke-based features, the key hold times are computed by performing an inner merge operation based on the key ID in press and release events in each window, whereby each press event will be joined with the nearest subsequent release event based on the key ID in each window. In addition, the system can handle cases where there are several presses of the same key before a release event occurs. Inter-key delay time is computed based on the differences in the timestamp of two subsequent keystrokes without any concern about the particular key. This operation uses NumPy vectorization techniques. Burst typing refers to consecutive keystrokes with intervals below a pre-specified limit, followed by counting and averaging the number of bursts.

Mouse-based features can be efficiently computed in a similar way. Mouse speed is computed using consecutive movement events and Euclidean distance equation:

$$v_i = \frac{\sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}}{t_{i+1} - t_i} \tag{6.1}$$

The computation of mouse speed requires the use of NumPy's `diff()` and `sqrt()` methods. In turn, mouse acceleration can be calculated based on the numerical differentiation of the speed values array. Several other error-specific features can be derived from the logs of user interactions. The backspace rate feature can be calculated by dividing the number of keypresses with either `Key.backspace` or `Key.delete` keys by the total number of keypresses recorded in the window. The feature of error correction is computed based on the number of sequences with backspace events followed by character input.

6.5 MACHINE LEARNING MODEL IMPLEMENTATION

The training interface that trains the models in question uses a common interface for all types of models we support in this project. The first step in the pipeline entails reading the feature set from the `final_dataset.csv` file via Pandas and splitting the feature set into the feature matrix **X** and the labels **y**. We proceed to use Scikit-learn's `train_test_split` function to perform stratified train-test splits. Then, we fit a `StandardScaler` on the training features and scale both training and testing sets. Finally, the hyperparameterized model is instantiated, trained on the training set, and tested on the test dataset. The results include classification reports and ROC-AUC score from Scikit-learn. We then save the model and scaler using Joblib's `dump`.

The implementations of the deep learning algorithms are based on the subclasses of `nn.Module` from PyTorch. In the process of training, shuffling is done with the use of the DataLoader, where the default value of batch size is equal to 32. Next, for each batch, a forward pass through the model is performed to get the prediction, and the binary cross-entropy loss between the prediction and actual label is computed, and a backward pass is used for computing the gradient. Optimization of the model is done by the Adam optimizer, and each epoch includes evaluating the performance of the model on the validation dataset. Early stopping technique is used whenever there is no improvement in validation loss for 15 consecutive epochs. After completing the training

process, the model is saved with the `torch.save` function.

6.6 LLM INTEGRATION

LLM integration is achieved using the OpenAI GPT API, which provides the generation of natural-language descriptions for the prediction results. The prompt fed into the model consists of the predicted state (relaxed or fatigued), prediction accuracy, most significant features along with their values, and the difference between the current feature values and user's baseline. The prompt is structured to elicit a response describing the predicted state in layman's terms, providing insights into behavioral factors that have led to the prediction, and actionable tips on managing fatigue.

The module also features support for error handling in case of API errors. This is achieved by using a combination of exponential backoff with retrying and fallback to explain the problem using a message based on a template in case the API fails.

6.7 WEB INTERFACE IMPLEMENTATION

The front end of the system utilizes Flask framework while server side rendering is carried out using the Jinja2 templating engine. The web user interface uses HTML, CSS, and JavaScript to render information about the system's behavior. The interface contains a status indicator which will show either "Relaxed" (green) or "Fatigued" (red), alongside with the probability of such outcome. In addition to that, there is an indicator containing graphs which shows trend lines for particular behavioral traits, such as typing speed, mouse speed, idle time, and mistake rate. There is also a panel containing explanation and recommendations based on the LLM. A final element of the dashboard consists of a summary table with recent values of all 24 behavioral characteristics.

The interactive visualizations have been built using Plotly.js. They include the following types of charts: Line plot displaying time series behavioral data, Bar chart comparing various performance metrics such as accuracy, precision, recall, and F1 score among different classifiers, Heat map of the confusion matrix for the chosen classifier, Bar chart depicting the importance of features in relation to the RF classifier.

6.8 INTEGRATION AND DEPLOYMENT

The system has used a modular design where each stage from data collection to visualization has its own interface for executing its respective function. This will improve sustainability and ensure that every module works effectively in the overall process flow.

1. Begin recording the actions of the keyboard and mouse in the background.
2. Begin the Flask-based web server which would serve as an interface with the user.
3. Process the latest logs in order to predict user fatigue periodically.
4. In case fatigue is detected, launch the LLM process to provide reasons and recommendations.
5. Update the dashboard and store predictions for later analysis.

Regarding implementation, the whole solution was implemented as a standalone program using Python programming language which can be executed by the command line interface. The configuration variables used by the solution are defined in a JavaScript Object Notation (JSON) file, where a user can change the values for the monitor frequency, model selection, threshold value, and LLM.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 EXPERIMENTAL SETUP

This chapter presents the experimental outcomes from the analysis of the TypeMind architecture. All experiments were conducted on a PC equipped with an Intel Core i7-12700H processor, 16 GB RAM, and NVIDIA RTX 3060 GPU. The following software environment was employed: Python 3.10, Scikit-learn 1.2.2, Pytorch 2.0.1, and Flask 2.3.2. The dataset comprising 210 instances (105 relaxed and 105 fatigued) and 24 behavioral characteristics.

7.2 CLASSICAL MACHINE LEARNING MODEL RESULTS

Five classical ML models were trained and evaluated using 5-fold stratified cross-validation. Table 7.1 presents the classification performance of each model.

Table 7.1: Classification Performance of Classical ML M

[INFO] Captions
Rule: Caption numbering gap: expected 6, found 7
Expected: 6
Detected: 7
Confidence: 100.0%

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (	
RF	91.43 ± 2.14	90.87 ± 2.53	92.05 ± 1.98	91.45 ± 2.10	0.9582
SVM	88.10 ± 2.87	87.62 ± 3.14	88.67 ± 2.65	88.14 ± 2.82	0.9347
KNN	85.24 ± 3.42	84.76 ± 3.78	85.81 ± 3.21	85.28 ± 3.38	0.9124
DT	83.81 ± 3.95	83.24 ± 4.12	84.48 ± 3.67	83.85 ± 3.89	0.8876
LR	82.38 ± 3.18	81.90 ± 3.54	82.95 ± 2.98	82.42 ± 3.12	0.8735

The RF classifier yielded the best performance, achieving an accuracy rate of 91.43%, while the SVM classifier came second, attaining an accuracy rate of 88.10% and the KNN classifier was third with an accuracy rate of 85.24%. The DT and LR classifiers had relatively low accuracy rates, which were 83.81% and 82.38%, respectively.

Furthermore, the RF classifier also had the maximum ROC-AUC value of 0.9582, which indicates that the classifier has high discriminatory power irrespective of the threshold. In addition, the balanced values of precision and recall imply that there is no bias in favor of any particular class of users by the proposed model. In reality, this is

very important since false positives and false negatives carry costs associated with each case.

7.2.1 Confusion Matrix Analysis

Table 7.2 presents the confusion matrix for the RF model on the test set.

Table 7.2: Confusion Matrix for Random Forest Classifier

	Predicted Relaxed	**Predicted Fatigued**
Actual Relaxed	14	2
Actual Fatigued	1	15

From the confusion matrix, it is apparent that the RF classifier correctly classified 14 out of 16 relaxed observations and 15 out of 16 fatigued observations in the test fold. There are two cases where the classifier erroneously predicted the relaxed state as the fatigued state and vice versa. Therefore, there exists a slight bias towards predicting fatigued observations, which can be deemed tolerable due to its application in a safety context.

7.3 DEEP LEARNING MODEL RESULTS

Three deep learning models were trained and evaluated using the same data split and cross-validation strategy. Table 7.3 presents the results.

Table 7.3: Classification Performance of Deep Learning Models

Model	**Accuracy (%)**	**Precision (%)**	**Recall (%)**	**F1-Score (%)**	**ROC-AUC**
LSTM	89.52 ± 2.65	89.05 ± 2.89	90.10 ± 2.47	89.57 ± 2.61	0.9456
CNN	88.57 ± 2.98	88.14 ± 3.21	89.05 ± 2.78	88.59 ± 2.94	0.9389
Transformer	87.14 ± 3.34	86.67 ± 3.56	87.71 ± 3.12	87.19 ± 3.28	0.9278

The best resulted deep learning model was LSTM with 89.52%, then came the CNN with 88.57% and the last was the Transformer with 87.14%. The advantage of LSTM over other models was its ability to "remember" information through many gates in the model Hochreiter and Schmidhuber (1997). The CNN model scored a very similar

result since the interesting features are local pattern of elements and the model were succeeded using the convolution of the elements LeCun et al. (1998).

Although a competitive result, the Transformer did not outperform the LSTM and CNN models. This may be due to the small dataset size (210 samples) which is likely the Transformer architectures are not able to take full advantage of their ability to learn more intricate attention patterns Vaswani et al. (2017) due to the limited training corpus. With a larger and more diverse dataset, the Transformer results should improve considerably.

Interestingly, all the three deep learning approaches were found to have slightly lower accuracy compared to the RF approach. This finding is consistent with previous studies that indicate that in cases where there is insufficient data for training, ensemble classical approaches exhibit superior generalization ability compared to deep learning approaches Bishop (2006).

7.4 ENSEMBLE MODEL RESULTS

Two ensemble strategies were evaluated to determine whether combining multiple models improves classification performance. Table 7.4 presents the results.

Table 7.4: Classification Performance of Ensemble Models

Model	**Accuracy (%)**	**Precision (%)**	**Recall (%)**	**F1-Score (%)**	**ROC-AUC**
Voting Classifier	92.38 ± 1.87	91.95 ± 2.12	92.86 ± 1.74	92.40 ± 1.85	0.9645
Stacking Classifier	93.33 ± 1.65	92.86 ± 1.89	93.81 ± 1.54	93.33 ± 1.62	0.9712

Ensemble methods performed better than all individual models. The Stacking Classifier scored the highest among all classification methods, with 93.33% accuracy and an ROC-AUC score of 0.9712, which was 1.9 percent higher than the most accurate individual model (RF, 91.43%). The Voting Classifier also outperformed other classification methods.

The above results validate the efficiency of ensemble-based learning techniques in fatigue detection owing to the ability of the diversity among base classifiers to cover for

any weakness of each of the individual models Dietterich (2000); Zhou (2012). This is also evident by the better performance of the Stacking Classifier compared to the Voting Classifier.

7.5 IMPACT OF VAE DATA AUGMENTATION

The influence of data augmentation using VAE on model performance was assessed by comparing models trained using the original data set (containing 210 samples) with models trained using the augmented data set (410 samples total). The results obtained using the best performing models are presented in Table 7.5.

Table 7.5: Impact of VAE Data Augmentation on Model Performance

Model	**Accuracy (Original)**	**Accuracy (Augmented)**	**F1 (Original)**	**F1 (Augmented)**
RF	91.43%	93.17%	91.45%	93.20%
SVM	88.10%	90.24%	88.14%	90.28%
LSTM	89.52%	92.20%	89.57%	92.24%
Stacking	93.33%	95.12%	93.33%	95.16%

All the models were able to achieve positive performance gains using the VAE-enhanced dataset. For the Stacking Classifier, the accuracy obtained was 95.12%, which was an increase of 1.79 percentage points from the original accuracy score. The biggest relative improvement was observed in the case of the LSTM model at an increase of 2.68 percentage points.

The findings confirm the utility of augmentation using VAE for behavioral biometrics data. The artificial samples produced through VAE maintain the statistical characteristics of the original dataset while ensuring sufficient diversity in the data to enhance generalization capabilities Kingma and Welling (2014).

7.6 FEATURE IMPORTANCE ANALYSIS

The relative importance of each behavioral feature was assessed using the RF model's feature importance ranking. Table 7.6 presents the top 10 features ranked by importance.

Table 7.6: Top 10 Features by Importance (Random Forest)

Rank	Feature	Importance (%)
1	avg_key_hold_time	14.23
2	typing_speed	12.87
3	avg_inter_key_delay	11.45
4	avg_mouse_speed	9.62
5	backspace_rate	8.34
6	idle_duration_total	7.91
7	pause_frequency	6.78
8	error_correction_rate	5.43
9	std_key_hold_time	4.89
10	avg_pause_duration	4.21

The feature importance plot shows keystroke dynamics features as the top three most important features, with `avg_key_hold_time` as the most important feature (14.23%), followed by `typing_speed` (12.87%) then `avg_inter_key_delay` (11.45%). This is in line with the literature where queuing "typing patterns [have been] found to be highly sensitive to the details of a person's cognitive state" Epp et al. (2011); Cao et al. (2020).

The appearances of `avg_mouse_speed` (9.62%) and `backspace_rate` (8.34%) in the top 5 show that mouse dynamics and error measures make desirable additional sources of parallel information. The features `idle_duration_total` (7.91%) (6.78%) indicate an increased pause rate and held attention diversion experiences mental fatigue.

[MINOR] Page Layout
Rule: Right margin overflow in BODY_TEXT.
Expected: <= 523.276pt
Detected: 549.17pt
Confidence: 77.7%
Reason: Single line overflow.

The findings regarding the feature importance reinforce the multi-modal approach used in the development of the TypeMind system because they reveal that all the mentioned features together provide a complete behavioral pattern to detect fatigue.

7.7 COMPARATIVE ANALYSIS

Table 7.7 presents a comparison of all models tested in this study based on their performance on the raw dataset.

Table 7.7: Comprehensive Model Comparison

Rank	Model	Accuracy	F1-Score	ROC-AUC	Training Time	Category
1	Stacking Classifier	93.33%	93.33%	0.9712	4.2s	Ensemble
2	Voting Classifier	92.38%	92.40%	0.9645	2.8s	Ensemble
3	Random Forest	91.43%	91.45%	0.9582	1.5s	Classical ML
4	LSTM	89.52%	89.57%	0.9456	45.3s	Deep Learning
5	CNN	88.57%	88.59%	0.9389	38.7s	Deep Learning
6	SVM	88.10%	88.14%	0.9347	0.8s	Classical ML
7	Transformer	87.14%	87.19%	0.9278	62.1s	Deep Learning
8	KNN	85.24%	85.28%	0.9124	0.2s	Classical ML
9	Decision Tree	83.81%	83.85%	0.8876	0.3s	Classical ML
10	Logistic Regression	82.38%	82.42%	0.8735	0.5s	Classical ML

It is evident from the above discussion that there are some obvious trends in this regard. For example, the ensemble techniques outperformed all other individual classification techniques, and the first two were the Stacking and Voting classifiers. It implies that ensembling can be considered effective to compensate for individual classifier shortcomings.

Another interesting discovery was the fact that classic machine learning algorithms were still highly competitive against deep learning algorithms in this study. In particular, RF classification showed higher accuracy levels than any deep learning classifier and demonstrated that classic ensemble algorithms could prove to be quite efficient in cases where datasets do not include a lot of data samples. Furthermore, classic machine learning algorithms needed significantly less training time than deep learning classifiers (from 38 to 62 seconds compared to 0.2 to 4.2 seconds).

Finally, the ROC-AUC scores for all models were high, with each model achieving an ROC-AUC score greater than 0.87. This suggests that although the rankings were different, all the models tested had some capacity to differentiate between the relaxed and fatigued classes.

7.8 LLM EXPLANATION QUALITY

Qualitative evaluation of the LLM module involved analyzing the explanations produced for prediction instances. Below is one such example of the explanation generated

by the LLM module for a fatigued prediction:

> *"Based on your current typing/mouse activity, we can determine that you may be at the point where your mental fatigue begins (accuracy: 87.3%). You are currently typing 23% slower than your normal pace, and also your hold rate of the keystrokes has increased to 18% slower than usual, which indicates slower response times. You are also making 35% more errors while typing as evident by your use of backspace. Your mouse movement has slowed by 15% and there have been more idle times. We recommend taking a 10–15 minute break from work."*

Interpretations generated by LLM offer context-rich, understandable interpretations of the change in numerical values of features into behavior-related information. Personalization occurs because of the unique behavior patterns identified through these interpretations.

7.9 SYSTEM PERFORMANCE

The real-time performance of the TypeMind system was evaluated in terms of prediction latency and resource utilization. Table 7.8 presents the system performance metrics.

Table 7.8: System Performance Metrics

Metric	**Value**
Feature extraction time (per window)	0.15 seconds
Model inference time (RF)	0.02 seconds
Model inference time (LSTM)	0.08 seconds
LLM explanation generation time	1.2–2.5 seconds
Total prediction pipeline latency	0.17–2.67 seconds
CPU utilization (logging)	1–3%
Memory utilization (total system)	250–400 MB
Log file growth rate	$\sim$2 MB/hour

The operation of the TypeMind system is performed effectively and instantly. In the case of classical machine learning models, the whole pipeline of prediction with feature

extraction and inference is completed within less than 0.25 seconds, which enables efficient system performance. When the LLM module starts working, there is an additional time needed for explaining predictions that amounts to 1.2-2.5 seconds. Nevertheless, this additional time is considered appropriate since explanations are provided only in cases of recognizing fatigue events but not after every prediction cycle completion. Moreover, the logging of background processes requires very little computing power of 1-3%.

7.10 DISCUSSION

The results confirmed our hypothesis, we are able to accurately predict users' mental fatigue using only their keyboard and mouse data. In general, we have demonstrated that fatigue actually manifests in ways that measurable quantifiable changes to user behavior. Our feature importance analysis reinforces what has already been established in current literature, that fatigued users hold keys down longer, type more slowly, pause more often, exhibit larger interkey gaps, move their mouse more slowly, and make more mistakes Boksem et al. (2005); Lorist et al. (2005).

The other crucial finding from this study is the usefulness of the multimodal approach in analyzing keystrokes. While keystroke-based attributes performed best overall, there was valuable data gained by incorporating attributes based on mouse operations that would have been otherwise unavailable if only keystrokes were considered. This demonstrates the need to include mouse activities as well in the detection of fatigue.

Moreover, the findings reveal a pragmatic compromise between model complexity and data volume. Although the deep learning models can be regarded as the best fit for time series data, it is obvious that the RF models and ensembles provided better results for the smaller dataset employed in the current research. Therefore, it can be argued that when data availability is restricted, classical ML models and ensembles might be more beneficial options. From a practical standpoint, the RF and the Stacking Classifier prove themselves to be the most effective choices due to their high efficiency.

Moreover, the influence of augmentation using VAE was clearly beneficial. The performance of all three models increased when additional synthetic data was introduced to the dataset, and the advantage of deep learning models became more pronounced thanks to the increase in the number of training examples. Thus, generative augmentation is shown to be a useful approach in the context of behavioral biometric recognition problems, where large labeled datasets may be difficult to acquire.

Moreover, the incorporation of explanations from LLM makes the system's predictions even more meaningful. Instead of being merely a black box, the system can provide explanations about its fatigue detection based on behavioral cues. From what we have observed, the generated explanations seem to be contextual, personalized, and practical, potentially enhancing user acceptance of the system.

Although these results are promising, there are a few caveats to keep in mind. The relatively small sample size of 210 users may constrain the generalization of the models. Since the data collection sessions were only 48 to 72 hours long, there could be user-specific differences in mouse and keyboard behavior that impact the transferability of the models. The investigation only viewed mental fatigue as a two-state variable (i.e., relaxed or mentally fatigued) while in reality, there are multiple degrees of mental fatigue. Finally, the quality of the LLM generated explanations was only examined qualitatively, not through a user study.

CHAPTER 8

CONCLUSION AND FURTHER ENHANCEMENTS

8.1 CONCLUSION

This paper presents TypeMind, a framework for non-intrusive detection of mental fatigue from behavioral data observed from computer keyboard and mouse activity. The approach presented in this paper is well motivated by the requirements to unobtrusively and continuously monitor the cognitive state during a period of time during computer usage.

Among the prime achievements of this thesis is the development of a non-intrusive data collection system. The framework records keystrokes and mouse events from the user efficiently as a Light-weight User-mode Program by utilizing the `pynput` library which is incorporated in the framework. It grabs mouse and keyboard events in real-time with negligible overhead (1–3% of CPU load). This high-precision data collection is appropriate for real-time behavior monitoring.

Another main contribution is the feature engineering. In total 24 features were defined and extracted in four broad areas, including key stroke behavior, mouse behavior, idle time and error-related characteristics. The features are related to key hold time, inter-key interval, mouse idle time, mouse speed and acceleration, mouse movement, idle time, backspace rate and so on. It is concluded that key hold time, typing speed and inter-key interval are most informative features for mental fatigue detection.

In addition, the study also proved that comparing multiple classification approaches could be powerful. Comparing 10 models on the 3 groups of methods: classical machine learning models, deep learning models and ensemble models, the Study found that, with the original data set, Stacking Classifier reached 93.33% accuracy and with VAE enabled augmentation, reached 95.12% accuracy. Results show that the ensemble learning is strong for behavioral fatigue Recognition tasks.

Furthermore, the VAE-based generative data augmentation module was effective in enhancing the overall model performance. The generated behavioral samples maintained the characteristics of the data and enhanced the diversity of data samples, leading to architecture-wide accuracy improvements of 1.74% to 2.68%.

Explainability is another important feature of this work. Since LLM-based explainability is employed to translate the model‘s numerical prediction results into comprehensible descriptions of behavioral change and actionable suggestions for fatigue management, the prediction process can be made more interpretable for better practical benefit.

The real-time web interface implemented using Flask serves to enhance the usability of TypeMind. The dashboard offers an easy interface to view the fatigue status, observe behavioral changes and view predictions along with the explanation provided by the LLMs.

This paper shows that, in general, keyboard and mouse use carries enough information to accurately assess mental fatigue with an accuracy of over 93% through ensemble classifiers. TypeMind is therefore a promisingand low-costsystem for fatigue detection given its practicality and respect for user privacy. A modular structure can further make it adaptable and expandable for other uses.

8.2 ACHIEVEMENTS AGAINST OBJECTIVES

What follows summarizes in Table 8.1, the objectives that have been achieved throughout the different projects carried out in chapter 2.

8.3 LIMITATIONS

Although the performance of TypeMind was promising, there are some caveats to the results that should be acknowledged when concluding on the system‘s applications. For one, the size of the dataset used to build the classifiers is relatively small. 210 observations collected through controlled experimental procedures is enough to do proof of

Table 8.1: Achievement of Project Objectives

No.	Objective	Status	Evidence
1	Design and develop a non-intrusive data acquisition module	Achieved	Chapter 6, Section 6.2
2	Implement a comprehensive feature extraction pipeline	Achieved	Chapter 6, Section 6.4
3	Train and evaluate multiple ML and DL models	Achieved	Chapter 7, Sections 7.2–7.4
4	Implement VAE-based data augmentation	Achieved	Chapter 7, Section 7.5
5	Integrate an LLM module for explanations	Achieved	Chapter 7, Section 7.7
6	Develop a web-based GUI	Achieved	Chapter 6, Section 6.7
7	Evaluate using standard classification metrics	Achieved	Chapter 7, All sections

concept but not too much to increase the robustness of the models or make the system more broadly applicable.

Another constraint of the system is the fact that at the moment is individual-system-oriented. The system is designed based on sessions data that are only collected per user. The trained models are specific for these existing users only, and could not be generalized for new users.

The two-class nature of this classification system limits the expressiveness of the outcome as well. Our implementation presently only classifies between relaxed and tired conditions. In reality, fatigue lies on a continuum, and a multi-class classifier or regression model may give a more informative and useful measure of the severity of an individual‘s mental fatigue.

Finally, it is likely that user behavior could vary depending on the task in hand. Tasks that involve more mousey input may exhibit different patterns of behavior than those that are more keyboard dominated. The current system does not explicitly factor in the task context as a variable.

Another limitation is the LLM explanation module. The explanation system currently requires online use of API models. Where there is no internet access available, the framework has to fall back onto the template-based explanations, which are less personally relevant and offer poorer contextual explanations.

Another important challenge is the ground truth labeling. Since they are based on the experimental protocol and the subjective self-assessment, the ground truth labeling might have some bias or miss the variability. To address this type of problems, physiological validation (e.g. correlating the ground truth with EEG) should be used.

8.4 FURTHER ENHANCEMENTS

Future work can be directed toward developing the TypeMind framework in additional ways to optimize its flexibility, fidelity, usability, and scientific rigor while generalizing it to more users and hardware/software platforms. Addressing multi-user adaptation presents one promising avenue for extension that would realize this goal. Applying user-adaptive modeling as is being developed in transfer learning Pan and Yang (2010) or few-shot learning would enable TypeMind to learn a new user from minimal calibration data (e. G.,12 tions). Using a pretrained model as a baseline and differentially updating it with a small amount of user-specific data can increase robustness across varying user types.

One other improvement would be to go out of binary classifications and have the fatigue represented by a continuous or ordinal scale like from 0 to 10. This would have been even more informative by having different levels of fatigue as to be.

Than only differentiating between relaxed and tired. This notion can be taken further, by adapting the task such that it becomes a regression task or an ordinal classification task with levels like alert, low fatigue, moderate fatigue and high fatigue. Future prototypes of this system could further exploit the current task context. If the framework had an understanding of when the user were programming, communicating, editing, surfing the web, it could create task specific behavior baseline levels and interpret fatigue within the context of the current task. This would allow the system to take into account that different applications require the different keyboard and mouse behavior.

A second promising avenue involves the addition of other sensing modalities. For example, webcam eye tracking (PERCLOS or blinking rate), light sensors, and application use data could provide complementary indicators of user state to the behavioral indicators gleaned from keystrokes and mouse clicks. Sensor fusion might then increases

detection accuracy without sacrificing the system‘s non-intrusive interface. Likewise, extending the system to monitor users over minutes, days, weeks, or months might enable investigators to see a panoramic view of longerterm fatigue trends. This could contribute to an understanding of emerging fatigue trends, fluctuations due to daily work schedules, and effects of circadian rhythms.

Federated learning is another promising future direction because it would enable learning from many users without requiring personal raw data to leave their machines Li et al. (2017). The concept here is that each devices data remains stationary at that device, and only gradients/updates are transferred to a hosting server (or clouds). This method could potentially improve generalization of the models while still preserving user privacy. Putting the TypeMind system on mobile and cross-platform devices would make it even more applicable in the real-world. Additional tablet and smart phone support, and functionality based on tap duration, swipe motion, and keyboard accuracy may help extend fatigue monitoring outside of desktops. A cross-platform implementation would also help keep track of a ubiquitous user as they move through different devices over the course of their day.

Intervention in real time could also increase the practical value of the system. For instance, the framework could suggest temporary breaks based on the users current fatigue state, could be integrated with other productivity tools to offer cues for task switching, or implementation might include game-like elements to encourage healthier work practices or environmental cues (for example, changes in lighting or music according to user fatigue levels). Another potential feature of realizable value that would be meaningful is the addition of offline LLM capability. This would involve substituting the cloud LLM API with a lightweight locally deployed LLaMA-based, distilled or otherwise, LLM that retains the context-aware explanation generation while also improving on the value of privacy, dependency on my-company.com server, and potentially reducing cost in the long-term.

Lastly, formal clinical validation may be considered to establish a stronger scientific basis for the framework. Research studies that involve physiological measurements such as EEG could offer more solid evidence for a behavioral-based assessment of

fatigue. A supervised study that compares the predictions of TypeMind against physiological measures of fatigue would better determine the system‘s accuracy and practical significance. In summary, TypeMind has an excellent basis for behavioral-based detection of mental fatigue. The future improvements discussed above would make it a more adaptive, intelligent, and scalable cognitive health platform to serve many more users on many more devices.

BIBLIOGRAPHY

Ahmed, A. A. E. and Traore, I. (2007). A new biometric technology based on mouse dynamics. *IEEE Transactions on Dependable and Secure Computing*, 4(3):165–179.

Bailey, K. O., Okolica, J. S., and Peterson, G. L. (2014). User identification and authentication using multi-modal behavioral biometrics. *Computers & Security*, 43:77–89.

Banerjee, S. P. and Woodard, D. L. (2012). Biometric authentication and identification using keystroke dynamics: A survey. *Journal of Pattern Recognition Research*, 7(1):116–139.

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Boksem, M. A., Meijman, T. F., and Lorist, M. M. (2005). Effects of mental fatigue on attention: an erp study. *Cognitive Brain Research*, 25(1):107–116.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901.

Cao, L., Li, Y., and Zhang, M. (2020). Keystroke dynamics for cognitive fatigue detection. *Computers in Human Behavior*, 112:106452.

Chandrashekar, G. and Sahin, F. (2014). A survey on feature selection methods. *Computers & Electrical Engineering*, 40(1):16–28.

Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM.

Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3):273–297.

Desmond, P. A. and Hancock, P. A. (2009). Active and passive fatigue states. *Stress, Workload, and Fatigue*, pages 455–465.

Devlin, J., Chang, M.-W., Lee, K., and Tuptanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, pages 4171–4186.

Dietterich, T. G. (2000). Ensemble methods in machine learning. *International Workshop on Multiple Classifier Systems*, pages 1–15.

Doersch, C. (2016). Tutorial on variational autoencoders. *arXiv preprint arXiv:1606.05908*.

Epp, C., Lippold, M., and Mandryk, R. L. (2011). Identifying emotional states using keystroke dynamics. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 715–724.

Grandjean, E. (1979). Fatigue in industry. *British Journal of Industrial Medicine*, 36(3):175–186.

Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media, 2nd edition.

Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.

Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the International Conference on Machine Learning*, pages 448–456.

Ji, Q., Zhu, Z., and Lan, P. (2004). Real-time eye, gaze, and face pose tracking for monitoring driver vigilance. *Real-Time Imaging*, 10(6):357–375.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of the International Conference on Learning Representations*.

Kingma, D. P. and Welling, M. (2014). Auto-encoding variational Bayes. *Proceedings of the International Conference on Learning Representations*.

Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. *Proceedings of the International Joint Conference on Artificial Intelligence*, 14(2):1137–1145.

Lal, S. K. and Craig, A. (2003). A critical review of the psychophysiology of driver fatigue. *Biological Psychology*, 55(3):173–194.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324.

Lorist, M. M., Boksem, M. A., and Ridderinkhof, K. R. (2005). Impaired cognitive control and reduced cingulate activity during mental fatigue. *Cognitive Brain Research*, 24(2):199–205.

Monrose, F. and Rubin, A. D. (2000). Keystroke dynamics as a biometric for authentication. *Future Generation Computer Systems*, 16(4):351–359.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8026–8037.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al. (2011). Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830.

Powers, D. M. W. (2011). Evaluation: from precision, recall and f-measure to roc, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1):37–63.

Pusara, M. and Brodley, C. E. (2004). User re-authentication via mouse movements. *Proceedings of the ACM Workshop on Visualization and Data Mining for Computer Security*, pages 1–8.

Shorten, C. and Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):1–48.

Sokolova, M. and Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958.

Sun, D., Paredes, P., and Canny, J. (2016). Mouse movement as an indicator of attention and mental workload. *Proceedings of the ACM International Joint Conference on Pervasive and Ubiquitous Computing*, pages 1154–1159.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008.

Vizer, L. M., Zhou, L., and Sears, A. (2009). Automated stress detection using keystroke and linguistic features: An exploratory study. In *Proceedings of the International Conference on Document Analysis and Recognition*, pages 460–464. IEEE.

Zhang, W., Cheng, B., and Lin, Y. (2017). A new method for real-time drowsiness detection using facial features. *Accident Analysis & Prevention*, 100:72–81.

Zhou, Z.-H. (2012). Ensemble methods: Foundations and algorithms. *Chapman and Hall/CRC*.

Appendix A- Source Code

A.1 KEYBOARD LOGGER MODULE

```
import csv
import time
from pynput import keyboard

class KeyboardLogger:
    def __init__(self, log_file="keyboard_log.csv"):
        self.log_file = log_file
        self.writer = None
        self.file = None
        self._init_file()

    def _init_file(self):
        self.file = open(self.log_file, 'a', newline='')
        self.writer = csv.writer(self.file)
        if self.file.tell() == 0:
            self.writer.writerow(
                ['timestamp', 'key', 'event_type']
            )

    def on_press(self, key):
        try:
            key_str = key.char if hasattr(key, 'char')
                      else str(key)
        except AttributeError:
            key_str = str(key)
        self.writer.writerow(
            [time.time(), key_str, 'press']
        )
        self.file.flush()

    def on_release(self, key):
        try:
            key_str = key.char if hasattr(key, 'char')
                      else str(key)
        except AttributeError:
            key_str = str(key)
        self.writer.writerow(
            [time.time(), key_str, 'release']
        )
        self.file.flush()

    def start(self):
        listener = keyboard.Listener(
```

```
            on_press=self.on_press,
            on_release=self.on_release
        )
        listener.start()
        return listener
```

Listing 8.1: Keyboard Event Logger

A.2 MOUSE LOGGER MODULE

```
import csv
import time
from pynput import mouse

class MouseLogger:
    def __init__(self, log_file="mouse_log.csv"):
        self.log_file = log_file
        self.writer = None
        self.file = None
        self.last_pos = None
        self.min_distance = 5
        self._init_file()

    def _init_file(self):
        self.file = open(self.log_file, 'a', newline='')
        self.writer = csv.writer(self.file)
        if self.file.tell() == 0:
            self.writer.writerow([
                'timestamp', 'event_type', 'x', 'y',
                'button', 'pressed', 'dx', 'dy'
            ])

    def on_move(self, x, y):
        if self.last_pos:
            dx = x - self.last_pos[0]
            dy = y - self.last_pos[1]
            dist = (dx**2 + dy**2)**0.5
            if dist < self.min_distance:
                return
        self.last_pos = (x, y)
        self.writer.writerow([
            time.time(), 'move', x, y, '', '', '', ''
        ])
        self.file.flush()

    def on_click(self, x, y, button, pressed):
        self.writer.writerow([
```

```
            time.time(), 'click', x, y,
            str(button), pressed, '', ''
        ])
        self.file.flush()

    def on_scroll(self, x, y, dx, dy):
        self.writer.writerow([
            time.time(), 'scroll', x, y, '', '', dx, dy
        ])
        self.file.flush()

    def start(self):
        listener = mouse.Listener(
            on_move=self.on_move,
            on_click=self.on_click,
            on_scroll=self.on_scroll
        )
        listener.start()
        return listener
```

Listing 8.2: Mouse Event Logger

A.3 FEATURE EXTRACTION MODULE

```
import pandas as pd
import numpy as np

class FeatureExtractor:
    def __init__(self, window_size=300):
        self.window_size = window_size

    def extract_keystroke_features(self, kb_df):
        features = {}
        press = kb_df[kb_df['event_type'] == 'press']
        release = kb_df[kb_df['event_type'] == 'release']

        # Key hold times
        hold_times = []
        for key in press['key'].unique():
            p = press[press['key'] == key]['timestamp']
            r = release[release['key'] == key]['timestamp']
            for pt in p.values:
                rt = r[r > pt]
                if len(rt) > 0:
                    hold_times.append(rt.iloc[0] - pt)

        features['avg_key_hold_time'] = (
```

```
            np.mean(hold_times) if hold_times else 0
        )
        features['std_key_hold_time'] = (
            np.std(hold_times) if hold_times else 0
        )

        # Inter-key delays
        press_times = press['timestamp'].sort_values()
        delays = press_times.diff().dropna().values
        features['avg_inter_key_delay'] = (
            np.mean(delays) if len(delays) > 0 else 0
        )
        features['std_inter_key_delay'] = (
            np.std(delays) if len(delays) > 0 else 0
        )

        # Typing speed (keys per minute)
        duration = (press_times.max() - press_times.min()
                    ) if len(press_times) > 1 else 1
        features['typing_speed'] = (
            len(press) / duration * 60
        )

        # Backspace rate
        bs_count = len(press[
            press['key'].str.contains('backspace|delete',
                                      case=False, na=False)
        ])
        features['backspace_rate'] = (
            bs_count / len(press) if len(press) > 0 else 0
        )

        return features

    def extract_mouse_features(self, ms_df):
        features = {}
        moves = ms_df[ms_df['event_type'] == 'move']

        if len(moves) > 1:
            dx = moves['x'].diff().dropna().values
            dy = moves['y'].diff().dropna().values
            dt = moves['timestamp'].diff().dropna().values
            dt[dt == 0] = 1e-6

            distances = np.sqrt(dx**2 + dy**2)
            speeds = distances / dt

            features['avg_mouse_speed'] = np.mean(speeds)
            features['std_mouse_speed'] = np.std(speeds)
```

```
            features['max_mouse_speed'] = np.max(speeds)
            features['total_mouse_distance'] = (
                np.sum(distances)
            )
        else:
            features['avg_mouse_speed'] = 0
            features['std_mouse_speed'] = 0
            features['max_mouse_speed'] = 0
            features['total_mouse_distance'] = 0

        return features
```

Listing 8.3: Behavioral Feature Extraction

A.4 MODEL TRAINING MODULE

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import (
    train_test_split, cross_val_score
)
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    classification_report, roc_auc_score
)
import joblib

class ModelTrainer:
    def __init__(self, dataset_path):
        self.df = pd.read_csv(dataset_path)
        self.scaler = StandardScaler()
        self.models = self._init_models()

    def _init_models(self):
        return {
            'RF': RandomForestClassifier(
                n_estimators=100, max_depth=10
            ),
            'SVM': SVC(
                kernel='rbf', probability=True
```

```
            ),
            'KNN': KNeighborsClassifier(n_neighbors=5),
            'DT': DecisionTreeClassifier(max_depth=8),
            'LR': LogisticRegression(max_iter=1000),
        }

    def train_and_evaluate(self, model_name):
        X = self.df.drop('label', axis=1).values
        y = self.df['label'].values

        X_train, X_test, y_train, y_test = (
            train_test_split(
                X, y, test_size=0.15,
                stratify=y, random_state=42
            )
        )

        X_train = self.scaler.fit_transform(X_train)
        X_test = self.scaler.transform(X_test)

        model = self.models[model_name]
        model.fit(X_train, y_train)

        y_pred = model.predict(X_test)
        report = classification_report(
            y_test, y_pred
        )
        auc = roc_auc_score(
            y_test, model.predict_proba(X_test)[:, 1]
        )

        joblib.dump(model, f'{model_name}_model.pkl')
        joblib.dump(self.scaler, 'scaler.pkl')

        return report, auc
```

Listing 8.4: Model Training Pipeline

A.5 FLASK WEB APPLICATION

```
from flask import Flask, render_template, jsonify
import joblib
import numpy as np

app = Flask(__name__)

model = joblib.load('fatigue_model.pkl')
```

```
scaler = joblib.load('scaler.pkl')

@app.route('/')
def dashboard():
    return render_template('dashboard.html')

@app.route('/predict', methods=['POST'])
def predict():
    features = extract_current_features()
    features_scaled = scaler.transform(
        [features]
    )
    prediction = model.predict(features_scaled)[0]
    confidence = model.predict_proba(
        features_scaled
    )[0].max()

    status = 'Fatigued' if prediction == 1 \
             else 'Relaxed'

    explanation = generate_llm_explanation(
        features, status, confidence
    )

    return jsonify({
        'status': status,
        'confidence': float(confidence),
        'explanation': explanation
    })

@app.route('/history')
def history():
    records = load_prediction_history()
    return render_template(
        'history.html', records=records
    )

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

Listing 8.5: Flask Web Application Routes